



COMSATS UNIVERSITY ISLAMABAD
ABBOTTABAD CAMPUS, PAKISTAN

**CYBERSENTINEL AI – INTELLIGENT CYBERSECURITY TOOLKIT
FOR PHISHING, MALWARE, AND THREAT DETECTION**

BY

MOHAVIA ARIF

CIIT/FA22-BCS-084/ATD

ANAS BASHIR

CIIT/FA22-BCS-081/ATD

ABDUL SAMAD PARACHA

CIIT/FA22-BCS-056/ATD

SUPERVISOR

DR. MAZHAR ALI

BACHELOR OF SCIENCE IN COMPUTER SCIENCE (2022–2026)



COMSATS UNIVERSITY ISLAMABAD
ABBOTTABAD CAMPUS, PAKISTAN

**CYBERSENTINEL AI – INTELLIGENT CYBERSECURITY
TOOLKIT FOR PHISHING, MALWARE, AND THREAT
DETECTION**

**A PROJECT PRESENTED TO
COMSATS UNIVERSITY ISLAMABAD, ABBOTTABAD CAMPUS**

**IN PARTIAL FULFILLMENT
OF THE REQUIREMENTS FOR THE DEGREE OF**

BACHELOR OF SCIENCE IN COMPUTER SCIENCE (2022–2026)

BY

MOHAVIA ARIF
CIIT/FA22-BCS-084/ATD

ANAS BASHIR
CIIT/FA22-BCS-081/ATD

ABDUL SAMAD PARACHA
CIIT/FA22-BCS-056/ATD

DECLARATION

We hereby declare that this project titled “CyberSentinel AI – Intelligent Cybersecurity Toolkit for Phishing, Malware, and Threat Detection” is the result of our own efforts and has not been copied from any external source. All sources of information used in this project have been properly acknowledged and referenced.

We further declare that this project report has not been submitted, either in whole or in part, for any other degree or qualification at this or any other institution.

If any part of this work is found to be plagiarized or reproduced from another source without proper citation, we accept full responsibility and the consequences as determined by the university.

MOHAVIA ARIF

ANAS BASHIR

ABDUL SAMAD PARACHA

CERTIFICATE OF APPROVAL

It is to certify that the Final Year Project titled “**CyberSentinel AI – Intelligent Cybersecurity Toolkit for Phishing, Malware, and Threat Detection**” has been developed by **Mohavia Arif (CIIT/FA22-BCS-084/ATD)**, **Anas Bashir (CIIT/FA22-BCS-081/ATD)**, and **Abdul Samad Paracha (CIIT/FA22-BCS-056/ATD)** under the supervision of **Dr. Mazhar Ali**.

In our opinion, this project is fully adequate in scope and quality for the award of the degree of **Bachelor of Science in Computer Science (2022–2026)**.

Supervisor

External Examiner

Head of Department
(Department of Computer Science)

EXECUTIVE SUMMARY

Cybersecurity threats such as phishing emails, malicious URLs, and malware-infected files continue to affect individuals and small organizations. The risk is especially serious for non-technical users who may not have access to affordable security tools or the expertise needed to interpret conventional threat-intelligence reports.

CyberSentinel AI - Intelligent Cybersecurity Toolkit for Phishing, Malware, and Threat Detection addresses this problem through a unified web application. It allows users to analyze email text and supported documents, inspect URLs, and submit files for malware reputation analysis through a simple dashboard.

The email module applies Natural Language Processing with word-level and character-level TF-IDF features and a binary Logistic Regression classifier that predicts ham or spam. The interface maps ham to Safe and spam to Phishing. Visible HTTP/HTTPS links found in email text are analyzed separately. The URL module uses ten rule-based structural checks together with AbuseIPDB reputation data, while the malware module uses SHA-256 lookup and live analysis through VirusTotal [8], [14], [15].

The implementation uses a React frontend, a Flask backend, and scikit-learn for the phishing model. PostgreSQL provides persistent storage in the deployed environment, while SQLite remains the local-development fallback. The backend is deployed on Render and the frontend on Vercel; the Gmail Chrome extension connects to the same backend [9], [11], [12], [24], [33], [34].

Results are presented through color-coded verdicts, confidence values, threat indicators, and safety recommendations. Scan summaries are stored in the database, and PDF reports are generated in the browser from the current result. Email alerts are attempted only for configured high-risk outcomes and do not interrupt scanning when SMTP delivery fails.

CyberSentinel AI therefore demonstrates a practical AI-assisted cybersecurity prototype. It combines a trained phishing classifier with deterministic URL rules and external file-reputation services while clearly separating implemented capabilities from future security hardening and advanced threat-analysis work.

ACKNOWLEDGEMENT

All praise is to Almighty Allah, whose countless blessings, guidance, and wisdom enabled us to successfully complete this project.

We would like to express our sincere gratitude to our respected supervisor, **Dr. Mazhar Ali**, for his continuous support, valuable guidance, and constructive feedback throughout the development of this project. His encouragement and insightful suggestions played a vital role in shaping the direction and quality of our work.

We are also thankful to the faculty members of the Department of Computer Science, COMSATS University Islamabad, Abbottabad Campus, for providing us with the knowledge, resources, and academic environment necessary for completing this project successfully.

Furthermore, we extend our heartfelt appreciation to our parents and families for their unwavering support, motivation, and prayers, which have been a constant source of strength during our academic journey.

Finally, we would like to acknowledge everyone who directly or indirectly contributed to the completion of this project.

MOHAVIA ARIF

ANAS BASHIR

ABDUL SAMAD PARACHA

ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
CSV	Comma Separated Values
DB	Database
FR	Functional Requirement
ML	Machine Learning
NLP	Natural Language Processing
SVM	Support Vector Machine
TF-IDF	Term Frequency – Inverse Document Frequency
UI	User Interface
URL	Uniform Resource Locator
VT	VirusTotal

Table of contents

1. Introduction.....	1
1.1 Brief.....	1
1.2 Relevance to Course Modules.....	1
1.3 Project Background.....	2
1.4 Literature Review.....	3
1.5 Analysis from Literature Review.....	3
1.6 Methodology and Software Process Model.....	4
1.6.1 Rationale for Selected Methodology.....	4
2. Problem Definition.....	5
2.1 Problem Statement.....	5
2.2 Deliverables and Development Requirements.....	5
2.3 Current System.....	6
3. Requirement Analysis.....	7
3.1 Use Case Diagram.....	7
3.2 Detailed Use Cases.....	8
3.3 Functional Requirements.....	12
3.4 Non-Functional Requirements.....	13
3.5 UML Diagrams.....	14
4. Design and Architecture.....	14
4.1 System Architecture.....	15
4.2 Entity Relationship Diagram.....	16
4.3 Process Flow Diagram.....	17
4.4 Class and Module Diagram.....	19
4.5 Sequence Diagram.....	20
5. Implementation.....	21
5.1 Algorithm.....	21
5.1.1 Phishing Email Detection Algorithm.....	21
5.1.2 Embedded Link Scanning Algorithm.....	22
5.1.3 Email and Document File Upload.....	23
5.1.4 URL Threat Analysis Algorithm.....	23

5.1.5 Malware File Scanning Algorithm	24
5.1.6 VirusTotal Live Upload for Unknown Files.....	25
5.1.7 Scan History and Database Storage Algorithm	25
5.1.8 Report Generation Algorithm	26
5.1.9 Admin Panel Workflow.....	26
5.1.10 Email Notification Workflow	27
5.1.11 Gmail/Chrome Extension Workflow.....	27
5.1.12 Live Deployment Workflow.....	28
5.2 External APIs	29
5.3 User Interface	30
5.3.1 Login and Registration Interface	30
5.3.2 User Dashboard	31
5.3.3 Email Phishing Scan Interface.....	32
5.3.4 Email Result and Embedded URL Interface	32
5.3.5 URL Threat Analysis Interface.....	33
5.3.6 Malware File Scan Interface	34
5.3.7 Scan History Interface	35
5.3.8 Report Interface	36
5.3.9 Admin Panel Interface	37
5.3.10 Email Notification Output	38
5.3.11 Gmail/Chrome Extension Interface	39
5.4 Summary	40
6. Testing and Evaluation	41
6.1 Manual Testing.....	41
6.1.1 System Testing	42
6.1.2 Unit Testing	43
6.1.3 Functional Testing.....	47
6.1.4 Integration Testing.....	48
6.2 Automated Testing	49
6.2.1 Tools Used.....	50
6.2.2 Automated API Test Results	50
6.2.3 Model and Performance Evaluation	51

6.2.4 Security Testing.....	52
6.3 Summary	53
7. Conclusion and Future Work	54
7.1 Conclusion.....	54
7.2 Future Work	55
7.2.1 Larger Pakistani Cyber Threat Dataset.....	55
7.2.2 Deep Learning-Based Phishing Detection.....	55
7.2.3 Full Browser-Wide Extension	55
7.2.4 Mobile Application.....	55
7.2.5 Behavioral Malware Analysis	56
7.2.6 Enterprise Dashboard and SIEM Integration.....	56
7.2.7 Multi-Language Support.....	56
7.2.8 Two-Factor Authentication and Advanced Security	56
7.2.9 Continuous Model Retraining	56
7.2.10 Cloud Scalability and Load Balancing	56
7.3 Summary	56
8. References	58

Table of Figures

Figure 3. 1: Use Case Diagram	8
Figure 4. 1: System Architecture Diagram.....	16
Figure 4. 2: Entity Relationship Diagram.....	17
Figure 4. 3: Process Flow Diagram	18
Figure 4. 4: Class and Module Diagram	19
Figure 4. 5: Sequence Diagram.....	20
Figure 5. 1: Login and Registration Interface.....	30
Figure 5. 2: User Dashboard	31
Figure 5. 3: Email Phishing Scan Interface	32
Figure 5. 4: Email Phishing Scan Result.....	33
Figure 5. 5: URL Threat Analysis Interface.....	34
Figure 5. 6: Malware File Scan Interface	35
Figure 5. 7: Scan History Interface	36
Figure 5. 8: PDF Report Generation Option.....	37
Figure 5. 9: Admin Panel Interface	38
Figure 5. 10: Email Notification Output.....	39
Figure 5. 11: Gmail Extension Interface.....	40

Table of Tables

Table 3. 1: USE CASE	8
Table 3. 2: Use Case Description – Scan URL for Threats	10
Table 3. 3: Use Case Description – Malware File Detection	11
Table 3. 4: Functional Requirements	12
Table 3. 5: Non-Functional Requirements.....	13
Table 5. 1: Details of APIs Used in the Project	29
Table 6. 1: System Testing.....	42
Table 6. 2: Phishing Email Detection Unit Test Case	44
Table 6. 3: Embedded Link Scanning Unit Test Case	44
Table 6. 4: Email and Document File Upload Unit Test Case	45
Table 6. 5: URL Analyzer Unit Test Case	45
Table 6. 6: Malware File Scanner Unit Test Case.....	46
Table 6. 7: Authentication and Database Unit Test Case.....	46
Table 6. 8: Admin Panel and Email Notification Unit Test Case	47
Table 6. 9: Functional Testing	47
Table 6. 10: Integration Testing	49
Table 6. 11: Tools Used for Automated Testing.....	50
Table 6. 12: Automated API Test Results.....	50
Table 6. 13: Model and Performance Results	51
Table 6. 14: Security Test Results	52

1. Introduction

This chapter presents an overview of the project titled “**CyberSentinel AI – Intelligent Cybersecurity Toolkit for Phishing, Malware, and Threat Detection.**” It introduces the purpose, significance, and background of the system, along with its relevance to academic coursework. The chapter also includes a review of existing systems, analysis of their limitations, and the methodology adopted for developing the proposed solution.

1.1 Brief

CyberSentinel AI is a web-based cybersecurity platform that combines three implemented detection approaches: machine-learning classification for phishing email text, rule-based analysis for suspicious URLs, and VirusTotal-assisted reputation analysis for uploaded files.

Users can submit email text, supported email or document files, URLs, and malware-scan files. Email results are presented as Safe or Phishing, URL results as Safe, Suspicious, or Malicious, and file results using VirusTotal-derived verdicts such as Clean, Suspicious, Malicious, Unknown, or Pending.

The phishing module uses hybrid TF-IDF features and Logistic Regression. The URL module is deterministic rather than machine-learning based and combines ten structural rules with AbuseIPDB IP reputation. The file module computes SHA-256 hashes and uses VirusTotal for known-file lookup and, when necessary, live upload analysis [14], [15].

The primary objective of CyberSentinel AI is to provide an accessible, affordable, and user-friendly cybersecurity solution tailored for individuals and small organizations, particularly in regions where advanced security tools are not readily available.

1.2 Relevance to Course Modules

The development of CyberSentinel AI integrates knowledge and concepts from multiple courses studied during the Bachelor of Science in Computer Science program. It demonstrates the practical application of theoretical concepts in a real-world cybersecurity solution.

Key course contributions include:

- Machine Learning: Implementation and evaluation of a Logistic Regression classifier for binary phishing-email detection.

- **Natural Language Processing (NLP):** Text preprocessing, tokenization, and TF-IDF vectorization for analyzing email content.
- **Software Engineering:** Application of Software Development Life Cycle (SDLC), modular system design, and Agile development methodology.
- **Web Development:** Development of a responsive frontend using React.js and backend API integration using Flask.
- **Database Systems:** Storage of users and email, URL, and file scan summaries in PostgreSQL, with SQLite used as a local fallback.
- **Computer Networks & Security:** Understanding of cyber threats such as phishing, malware, and malicious URLs, and techniques for detecting and mitigating them.

This project effectively bridges academic learning with practical implementation, providing hands-on experience in designing and deploying an intelligent cybersecurity system.

1.3 Project Background

With the rapid growth of internet usage and digital services, cybersecurity threats have become increasingly prevalent. Users frequently encounter phishing emails, fraudulent websites, and malicious software designed to steal sensitive information or compromise systems.

In Pakistan, many individuals and small businesses lack access to reliable cybersecurity tools due to high costs, technical complexity, and limited awareness. As a result, users often rely on guesswork when dealing with suspicious content, leading to increased vulnerability to cyberattacks.

Existing solutions such as antivirus software and online scanning tools are often either too complex for non-technical users or require paid subscriptions. Additionally, many systems focus on a single type of threat and do not provide a unified platform for analyzing multiple types of cybersecurity risks.

CyberSentinel AI addresses these challenges through one web interface that combines an ML-based phishing classifier, rule-based URL scoring, and VirusTotal-assisted file analysis. This hybrid approach keeps each module aligned with the technique actually suited to its input.

1.4 Literature Review

Several existing systems and platforms have been developed to address cybersecurity threats such as phishing, malicious URLs, and malware detection. These systems provide valuable insights into threat detection techniques but also exhibit certain limitations.

VirusTotal is widely used to inspect files and URLs using results contributed by multiple security engines. It is effective for known-file reputation but its detailed results can be difficult for non-technical users to interpret [14].

PhishTank, Kaspersky Threat Intelligence Portal, and similar services provide useful phishing or malware intelligence, but they are separate platforms rather than a unified end-user workflow. PhishTank was reviewed during the project but is not integrated into the final implementation [16], [38].

Cisco Talos Intelligence provides broad threat-intelligence and network-security information, but its professional focus can be difficult for general users who only need a clear decision and recommended action [37].

Research and standard machine-learning libraries support classifiers such as Logistic Regression, Support Vector Machines, and transformer-based models for text classification. CyberSentinel AI selects Logistic Regression because it is fast, interpretable, and suitable for sparse TF-IDF features [4], [5], [6], [8].

Overall, while existing systems provide effective threat detection, they often lack accessibility, integration of multiple threat detection methods, and simplicity required for non-technical users.

1.5 Analysis from Literature Review

The analysis of existing systems reveals several key limitations that highlight the need for an improved cybersecurity solution. Most platforms focus on a single type of threat, such as file scanning or URL analysis, and do not provide a unified system for detecting multiple threats simultaneously.

Additionally, many existing tools are either too complex for non-technical users or require paid subscriptions, limiting their accessibility. The lack of user-friendly interfaces and clear result interpretation further reduces their effectiveness for everyday users.

A single detection technique is not sufficient for every threat type. CyberSentinel AI therefore uses a trained text classifier for email content, deterministic structural rules for URLs, and continuously updated external reputation data for files and resolved IP addresses.

The final system combines these techniques without claiming that every module uses machine learning. VirusTotal and AbuseIPDB provide live external intelligence, while the locally loaded phishing model provides low-latency text classification [14], [15].

Furthermore, the system emphasizes simplicity and accessibility by offering a user-friendly web interface with clear visual indicators and actionable insights. This makes it suitable for non-technical users while maintaining effectiveness in threat detection.

1.6 Methodology and Software Process Model

The development of CyberSentinel AI follows an Object-Oriented Methodology (OOM) combined with an Agile Software Development Life Cycle (SDLC). This approach enables modular development, flexibility, and continuous improvement throughout the project lifecycle.

The Object-Oriented Methodology is used to structure the system into independent modules such as phishing detection, URL analysis, malware scanning, and user management. Each module is designed as a separate component, ensuring scalability, maintainability, and ease of integration.

The Agile methodology is adopted to manage the development process through iterative and incremental cycles. The project is divided into multiple phases, including dataset preparation, model development, backend implementation, frontend integration, and testing. Each phase is developed and refined through continuous feedback and evaluation.

1.6.1 Rationale for Selected Methodology

The Object-Oriented approach is selected because it supports modular system design, allowing different components of the system to be developed and tested independently. It also enhances code reusability and simplifies system maintenance.

The Agile development model is chosen due to its flexibility and suitability for AI-based systems, where continuous refinement and testing are required. It allows the team to adapt to changes, improve model performance, and integrate new features efficiently.

Together, these methodologies ensure that CyberSentinel AI is developed as a scalable, efficient, and well-structured cybersecurity platform capable of evolving with future requirements.

2. Problem Definition

This chapter defines the core problem addressed by CyberSentinel AI and explains the limitations of current cybersecurity practices. It also outlines the deliverables of the proposed system and provides an overview of existing solutions.

2.1 Problem Statement

The increasing reliance on digital communication and online platforms has led to a significant rise in cybersecurity threats such as phishing attacks, malicious URLs, and malware infections. Users are frequently exposed to deceptive emails, fraudulent websites, and harmful files that are designed to steal sensitive information, including login credentials, financial data, and personal details.

In Pakistan, individuals and small businesses are particularly vulnerable due to limited cybersecurity awareness and lack of access to affordable and easy-to-use security tools. Many users are unable to distinguish between legitimate and malicious content, which results in increased risks of data breaches, financial losses, and system compromise.

Traditional cybersecurity solutions, including antivirus software and online scanning tools, are often complex, expensive, or designed for technically skilled users. These systems typically require installation, subscription fees, and prior knowledge, making them inaccessible to a large portion of the population. Moreover, most existing platforms focus on a single type of threat and fail to provide a unified solution for detecting phishing, malicious URLs, and malware in one system.

Therefore, there is a need for an intelligent, accessible, and user-friendly cybersecurity platform that can detect multiple types of threats in real time and present results in a clear and understandable manner for non-technical users.

2.2 Deliverables and Development Requirements

The key deliverables of CyberSentinel AI include:

- A fully functional web-based cybersecurity application

- Phishing email detection module using NLP and machine learning
- Malicious URL analysis module with risk scoring
- File malware detection module integrated with VirusTotal API
- User-friendly dashboard with visual threat indicators
- Report generation and download functionality
- Scan history and logging system
- Integration with implemented external services: VirusTotal, AbuseIPDB, and SMTP email delivery

The development requirements of the system include:

- Frontend development using React.js
- Backend API development using Flask
- Machine learning implementation using scikit-learn
- Natural Language Processing using TF-IDF techniques
- Integration of external APIs for enhanced threat detection
- Standard computing environment with internet connectivity
- Compatibility with modern web browsers (Chrome, Firefox, Edge)
- Secure communication using HTTPS protocols

2.3 Current System

Existing cybersecurity solutions primarily rely on traditional antivirus software and online scanning platforms. While these systems provide basic protection, they have several limitations in terms of usability, accessibility, and scope.

Limitations of Current Systems:

- Most tools focus only on malware detection and do not effectively handle phishing or malicious URLs
- Lack of a unified platform, requiring users to use multiple tools for different threats
- Complex interfaces that are difficult for non-technical users to understand
- Dependence on paid subscriptions, making them less accessible to general users
- Requirement of installation and system resources for antivirus software
- Limited real-time detection capabilities in some tools
- Difficulty in interpreting detailed technical reports provided by platforms like VirusTotal
- Lack of user-friendly visualization and simplified results
- Dependence on user judgment due to fragmented analysis

These limitations highlight the need for a unified, AI-driven cybersecurity solution like CyberSentinel AI, which provides real-time threat detection through a simple and accessible web interface.

3. Requirement Analysis

This chapter defines the functional and non-functional requirements of the CyberSentinel AI system. It explains how users interact with the system through use case modeling and describes the expected behavior of each module. The purpose of this chapter is to provide a clear and structured specification of system requirements for design and implementation.

3.1 Use Case Diagram

The use case diagram provides a high-level representation of the interactions between system actors and CyberSentinel AI. The primary actors include the **User** and **Admin**.

The User registers or logs in, performs email, URL, and file scans, reviews the latest scan records, downloads result reports, and may scan an open Gmail message through the extension. The Administrator can view system statistics, registered users, and recent scan records; user editing, settings management, and model management are outside the current prototype.

CyberSentinel AI - Implemented Use Cases

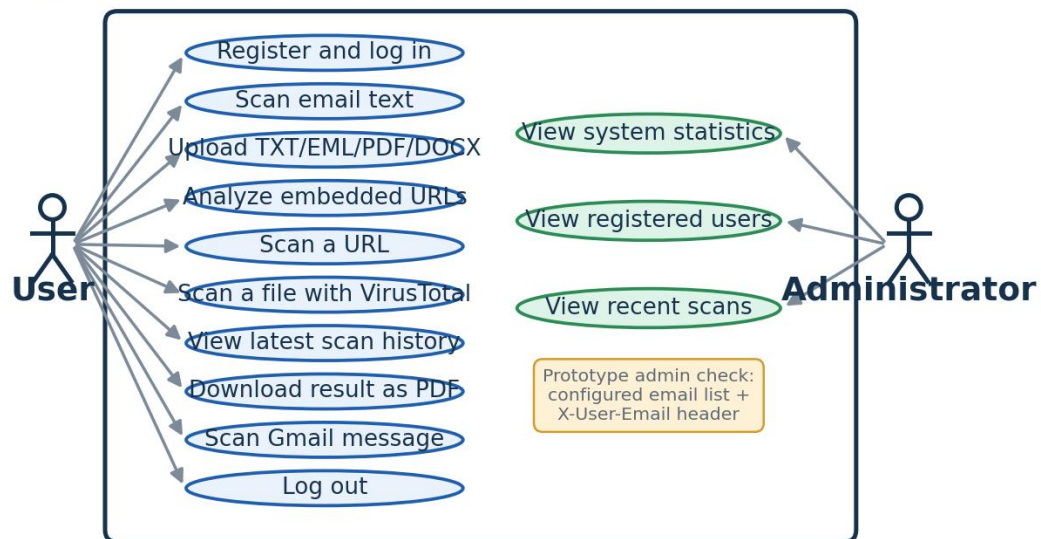


Figure 3. 1: Use Case Diagram

3.2 Detailed Use Cases

Table 3. 1: USE CASE

Field:	Description
Use Case ID:	UC-01
Use Case Name:	Scan Email for Phishing

Primary Actor:	User
Secondary Actors:	Backend API, NLP Module, ML Model
Description:	User submits email text for phishing detection
Trigger:	User clicks "Analyze"
Preconditions:	Internet available, system running, model loaded
Postconditions:	Prediction displayed, result stored in logs

Description – Scan Email for Phishing

Normal Flow:

1. User enters email text
2. User clicks analyze
3. System validates input
4. NLP preprocessing applied
5. ML model generates prediction
6. Result displayed with confidence score

Alternative Flow:

- Empty input → error message
- Server unavailable → failure notification

Table 3. 2: Use Case Description – Scan URL for Threats

Field	Description
Use Case ID	UC-02
Use Case Name	Analyze URL
Primary Actor	User
Secondary Actors	Backend API, rule-based URL analyzer, AbuseIPDB
Description	User submits URL for threat analysis
Trigger	User clicks "Analyze URL"
Preconditions	Internet connection, API keys active
Postconditions	Risk level displayed, result stored

Normal Flow:

1. User inputs URL
2. System validates format
3. Rule-based analyzer evaluates URL features

4. AbuseIPDB provides IP reputation when available
5. Risk score generated
6. Result displayed

Table 3. 3: Use Case Description – Malware File Detection

Field	Description
Use Case ID	UC-03
Use Case Name	File Malware Detection
Primary Actor	User
Secondary Actors	Backend API, VirusTotal API
Description	User uploads file for malware scanning
Trigger	User uploads file
Preconditions	Valid file, API access available
Postconditions	Malware result displayed

Normal Flow:

1. User uploads file
2. System validates file
3. File hash generated
4. API request sent
5. Results retrieved
6. Output displayed

3.3 Functional Requirements

The functional requirements define the expected behavior of the system.

Table 3. 4: Functional Requirements

ID	Requirement
FR-01	System shall allow users to submit email text
FR-02	System shall classify email using NLP and ML
FR-03	System shall allow URL submission
FR-04	System shall analyze URLs using rule-based scoring and optional AbuseIPDB reputation.
FR-05	System shall allow file uploads
FR-06	System shall integrate VirusTotal API

ID	Requirement
FR-07	System shall display results with color indicators
FR-08	System shall store scan history
FR-09	System shall generate downloadable reports
FR-10	System shall support admin functionalities

3.4 Non-Functional Requirements

Table 3. 5: Non-Functional Requirements

Category	Requirement
Usability	System shall provide simple UI for non-technical users
Performance	Response time should be within a few seconds
Security	Data must be transmitted via HTTPS

Category	Requirement
Reliability	System should handle failures gracefully
Maintainability	System should support modular updates

3.5 UML Diagrams

The system design is supported by UML diagrams to visually represent structure and behavior.

The following diagrams will be included:

- Use Case Diagram
- Class Diagram
- Sequence Diagram
- Activity Diagram
- Entity Relationship Diagram
- *CyberSentinel AI – Intelligent Cybersecurity Toolkit*

4. Design and Architecture

This chapter presents the overall system design and architecture of CyberSentinel AI. It explains the structural organization of the system and illustrates how different components interact to perform cybersecurity threat detection. The chapter also includes UML diagrams and database design used for system implementation.

4.1 System Architecture

CyberSentinel AI follows a modular client-server architecture comprising React and Chrome-extension clients, Flask API routes, detection services, persistent storage, and external intelligence services. The modules communicate through JSON or multipart HTTP requests.

The client layer contains the React web application and the Gmail Chrome extension. The web interface provides email, URL, file, history, report, and administrative views, while the extension submits the visible text of an opened Gmail message.

The Flask backend validates requests, applies rate limits and payload controls, invokes the appropriate detection service, returns JSON responses, stores scan summaries, and attempts configured email notifications. External integrations are limited to VirusTotal, AbuseIPDB, and SMTP [11], [14], [15], [26].

The service layer contains the phishing NLP/ML pipeline, the deterministic URLAnalyzer, the MalwareScanner, database utilities, and notification functions. Only the phishing email classifier uses machine learning; URL decisions are rule-based and malware verdicts are derived from VirusTotal responses.

The data layer uses PostgreSQL when DATABASE_URL is configured and SQLite otherwise. It stores users plus email_scans, url_scans, and file_scans records. Application log files and browser-generated PDF reports are not database entities.

CyberSentinel AI - Implemented System Architecture

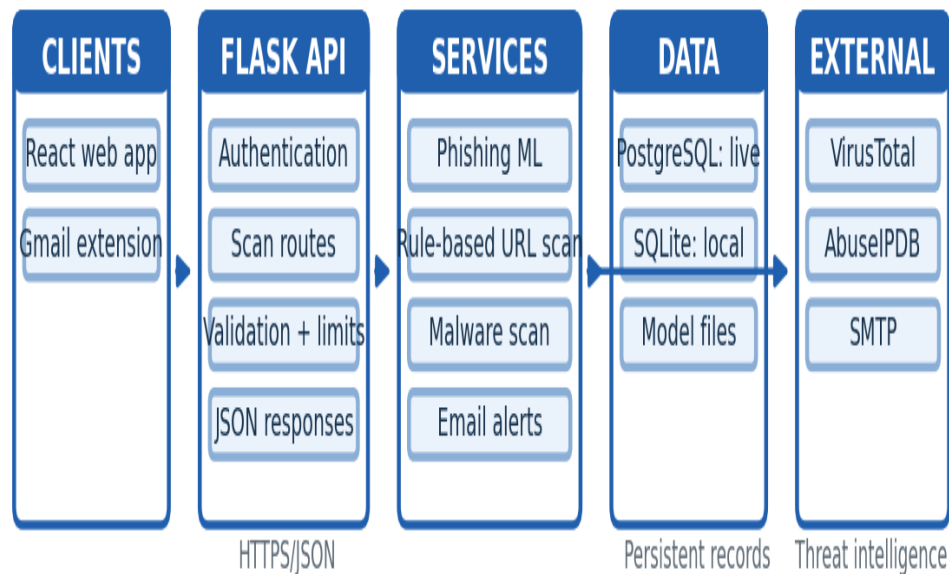


Figure 4. 1: System Architecture Diagram

4.2 Entity Relationship Diagram

The Entity Relationship diagram represents the four tables created by the current database initialization code: users, email_scans, url_scans, and file_scans.

Each scan table stores a user_email value together with module-specific result fields and a timestamp. These values provide a logical relationship to users.email, although the current schema does not enforce foreign-key constraints.

PostgreSQL is used for persistent live deployment and SQLite is retained for local development. Both database paths expose the same logical table structure.

CyberSentinel AI - Database Entity Relationship Diagram

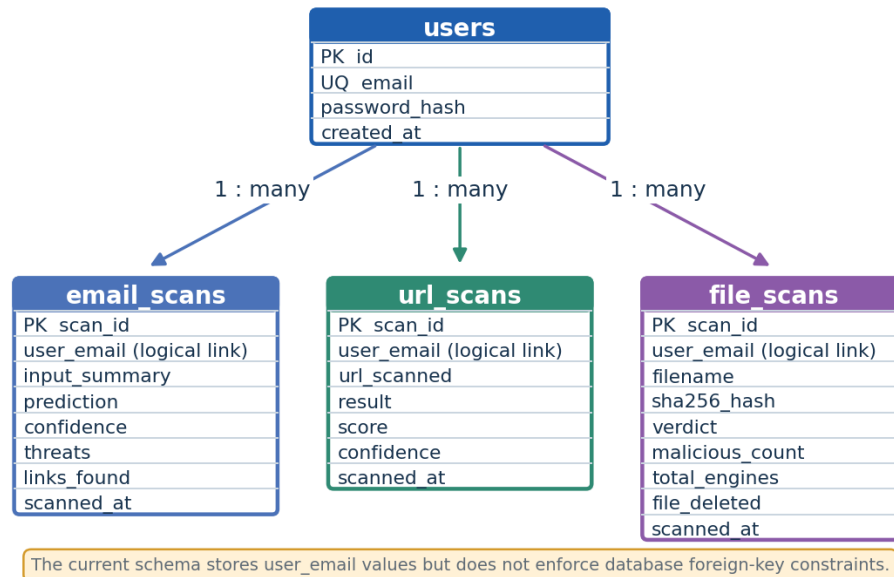


Figure 4. 2: Entity Relationship Diagram

4.3 Process Flow Diagram

The process flow diagram describes the implemented path from user input through validation, module-specific analysis, database storage, result presentation, and optional client-side PDF generation.

Email text and extracted document text pass through sanitization, hybrid TF-IDF transformation, and Logistic Regression; up to five unique visible URLs may also be analyzed. Submitted URLs use structural scoring and AbuseIPDB. Uploaded files use SHA-256 lookup and optional VirusTotal live upload.

After a scan, a summary record is stored in the appropriate scan table and the JSON result is displayed with verdicts, confidence values, indicators, and recommendations. The user can then download the current result as a PDF or open the latest-history view.

CyberSentinel AI - Implemented Process Flow

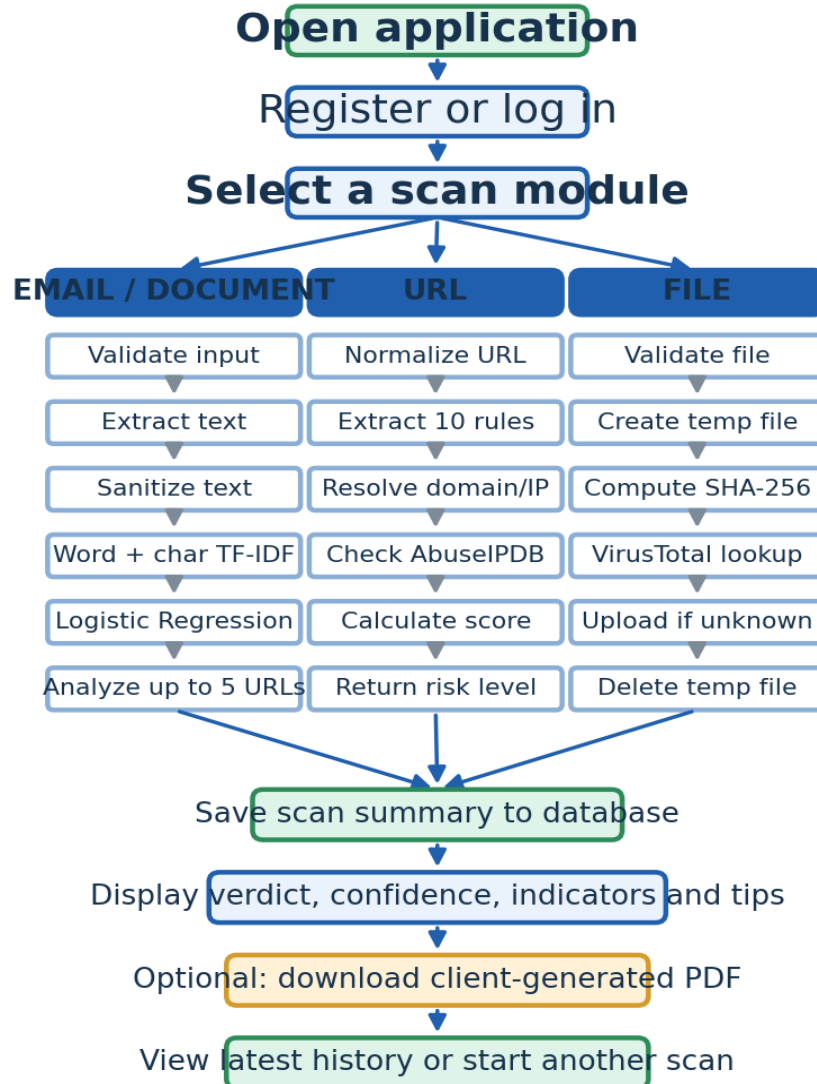


Figure 4. 3: Process Flow Diagram

4.4 Class and Module Diagram

The implementation is organized around Flask blueprints, two service classes, and supporting modules rather than separate domain classes for User, Admin, ScanSession, or Report.

URLAnalyzer and MalwareScanner are the principal service classes. Prediction, notification, database, cleaning, and URL-inspection behavior is provided by modules and functions.

The diagram shows how Flask routes invoke these services and how shared database and notification utilities support the request workflow.

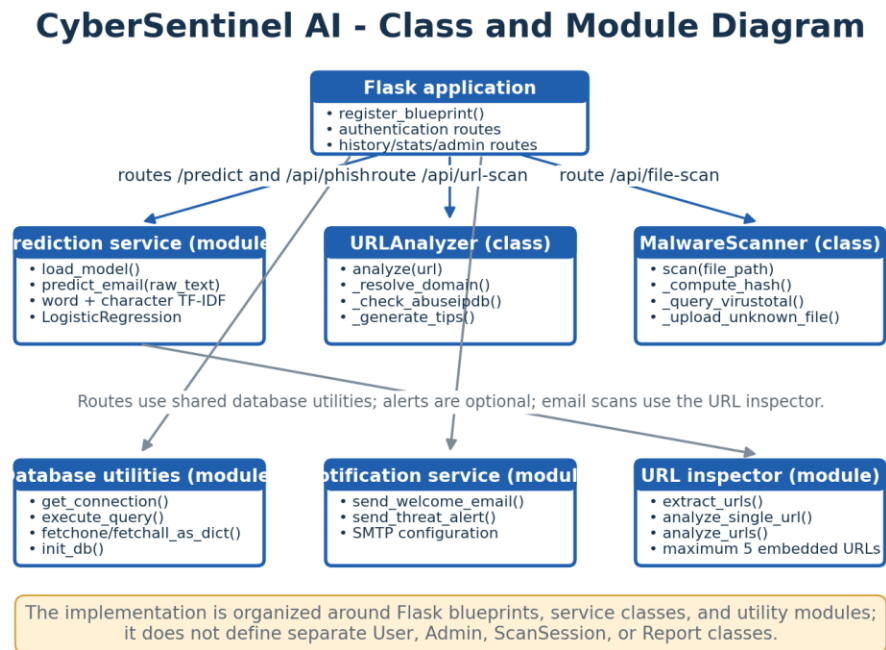


Figure 4. 4: Class and Module Diagram

4.5 Sequence Diagram

The sequence diagram illustrates communication among the user, React application or extension, Flask API, detection service, external intelligence provider, and database during a scan.

The frontend sends JSON or multipart data to Flask. Flask validates the request and invokes the phishing, URL, or file service. Optional AbuseIPDB or VirusTotal calls are made only by the relevant module. The result is stored as a scan summary and returned as JSON.

Threat-email delivery is attempted after selected high-risk results. Notification failure is logged and does not change the successful scan response.

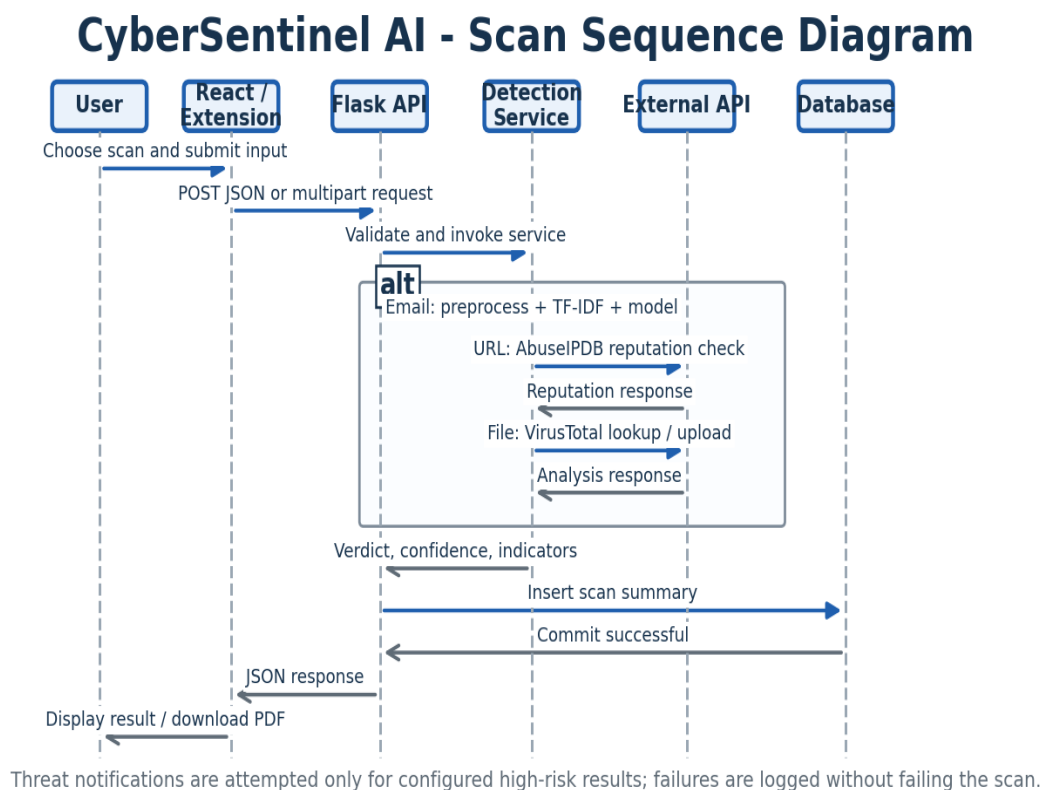


Figure 4. 5: Sequence Diagram

5. Implementation

This chapter explains the implementation details of CyberSentinel AI – Intelligent Cybersecurity Toolkit for Phishing, Malware, and Threat Detection. It describes the main algorithms used in the project, the external APIs integrated into the system, and the user interface screens developed for different users. The implementation of CyberSentinel AI is based on a modular approach in which each major functionality is developed as a separate component and then integrated into a unified web-based cybersecurity platform.

The final implementation includes binary phishing email detection, visible embedded-link analysis, TXT/EML/PDF/DOCX text input, rule-based URL analysis, VirusTotal file lookup and live upload, scan-summary storage, browser-generated PDF reports, prototype authentication and administration, SMTP notifications, a Gmail Chrome extension, public dashboard statistics, and live deployment. PostgreSQL is used live and SQLite is the local fallback.

The frontend handles interaction and report generation, Flask routes coordinate validation and services, the phishing pipeline performs ML classification, URLAnalyzer performs deterministic scoring, MalwareScanner communicates with VirusTotal, and four relational tables store users and scan summaries.

5.1 Algorithm

This section explains the major algorithms and workflows implemented in CyberSentinel AI. The purpose of this section is to describe how the main system modules operate without including source code. Each algorithm is explained using step-by-step natural language so that the working of the system can be clearly understood.

5.1.1 Phishing Email Detection Algorithm

The phishing email module analyzes text using Natural Language Processing and Machine Learning. Its Logistic Regression model is binary and predicts ham or spam; the interface maps these labels to Safe or Phishing. Word-level TF-IDF captures phrases, while character-level TF-IDF captures obfuscation and character patterns.

Algorithm Steps:

1. User enters email text or uploads an email file through the frontend.
2. The frontend sends the email content to the Flask backend.
3. The backend validates the input to ensure that the email text is not empty.
4. The input is sanitized to remove harmful HTML tags, scripts, and unnecessary characters.
5. Text preprocessing is applied, including lowercasing, punctuation removal, URL tokenization, and spacing normalization.

6. Word-level TF-IDF vectorization is applied to capture important phishing words and phrases.
7. Character-level TF-IDF vectorization is applied to capture obfuscated patterns.
8. Both feature matrices are combined into a single feature vector.
9. The trained Logistic Regression model predicts the class of the email.
10. The system calculates the confidence score and prepares threat indicators.
11. The final result is returned to the frontend and stored in scan history.
12. If the prediction is spam with confidence above 0.70, the notification module may attempt a phishing alert.

Output of the module includes:

- Binary classification result (ham/spam, displayed as Safe/Phishing)
- Confidence score
- Threat indicators
- Safety recommendations
- Prediction processing time in backend logs
- Scan history record

5.1.2 Embedded Link Scanning Algorithm

The embedded link scanning module extracts visible URLs from email text and analyzes each URL individually using the URL analyzer. This improves phishing detection because many phishing emails contain malicious links even when the surrounding message appears normal.

Algorithm Steps:

1. User submits email text for phishing analysis.
2. The backend scans the email text using a URL extraction pattern.
3. Up to five unique visible HTTP/HTTPS URLs are collected for analysis.
4. Duplicate URLs are removed to avoid repeated scanning.
5. Each extracted URL is passed to the URL analyzer module.
6. The URL analyzer checks structural URL features and reputation information.
7. Each link receives an individual verdict as Safe, Suspicious, or Malicious.
8. The embedded link results are combined with the main phishing email result.
9. The frontend displays a separate embedded links section.

10. The scan record is stored in the database.

5.1.3 Email and Document File Upload

The email page accepts TXT and EML files as decoded text input and also accepts PDF and DOCX documents for text extraction. The extracted text is sent through the same phishing pipeline. The current version does not parse MIME parts, inspect HTML anchor tags, or compare visible link text with hidden href destinations.

Algorithm Steps:

1. User uploads a TXT, EML, PDF, or DOCX file.
2. The backend validates the filename, extension, size, and non-empty content.
3. TXT and EML content is decoded using UTF-8 with a Latin-1 fallback.
4. PDF text is extracted page by page; DOCX text is collected from non-empty paragraphs.
5. Extracted text must contain at least ten readable characters.
6. At most the first 5,000 characters of document text are passed to the phishing classifier.
7. Visible HTTP/HTTPS URLs present in the extracted text may be analyzed separately.
8. The response includes the filename, extraction method, extracted character count, prediction, confidence, threats, tips, and embedded-link results.
9. A summary is stored in email_scans using the supplied user-email header or anonymous as fallback.
10. Image-only or unreadable documents return a clear extraction error rather than a fabricated prediction.

5.1.4 URL Threat Analysis Algorithm

The URL threat analyzer checks suspicious URLs using structural features and threat intelligence. It classifies URLs as Safe, Suspicious, or Malicious. The analyzer evaluates URL length, IP-based domains, @ symbols, HTTP usage, suspicious keywords, hyphens, shorteners, excessive subdomains, suspicious top-level domains, and long paths.

Algorithm Steps:

1. User submits a URL through the URL scan page or through an embedded email link.
2. The backend validates the URL format and normalizes it if necessary.
3. The system extracts structural URL features such as length, domain format, suspicious keywords, subdomain count, scheme, and path length.
4. Each suspicious feature contributes to a risk score.
5. The system resolves the domain to an IP address where possible.

6. The resolved IP is checked using AbuseIPDB.
7. The structural score and AbuseIPDB reputation value are combined when reputation data is available.
8. Thresholds convert the final score into Safe, Suspicious, or Malicious.
9. The response includes the score, confidence, threat indicators, AbuseIPDB status, and safety tips.
10. The result is displayed and may trigger an alert only when it is malicious with confidence above 0.60.
11. The URL scan summary is saved in url_scans.

Output of the module includes:

- Final URL verdict
- Risk score
- Matched suspicious features
- Threat intelligence result
- Recommendation for the user

5.1.5 Malware File Scanning Algorithm

The malware file scanning module allows users to upload suspicious files for scanning. The system validates the file, computes its SHA-256 hash, and checks it through VirusTotal. File scanning is designed with privacy in mind because files are handled temporarily and are not permanently stored.

Algorithm Steps:

1. User uploads a file through the file scan page.
2. The frontend sends the file to the Flask backend using multipart form data.
3. The backend validates the extension, non-empty content, and maximum size of 32 MB.
4. The file is temporarily saved in a secure temporary location.
5. The system calculates the SHA-256 hash of the file.
6. The file is checked against VirusTotal using hash lookup.
7. If VirusTotal returns a known result, the system parses malicious, suspicious, harmless, undetected, and flagged-engine counts.
8. The system generates a verdict as Clean, Suspicious, or Malicious.
9. The temporary file is deleted after the lookup or live-analysis workflow; the application does not retain the uploaded file.

10. The result is returned to the frontend and stored as scan metadata.
11. An email notification is attempted only when the final verdict is Malicious and at least one engine detected the file.

5.1.6 VirusTotal Live Upload for Unknown Files

A limitation of hash-based scanning is that a new file may not exist in the VirusTotal database. The final system improves this by uploading unknown files to VirusTotal for live analysis and polling until the result becomes available.

Algorithm Steps:

1. File hash lookup is performed through VirusTotal.
2. If the file is found, the system uses the returned report.
3. If the file is not found, the system marks it as unknown temporarily.
4. The backend uploads the file to VirusTotal's file analysis endpoint.
5. VirusTotal returns an analysis ID.
6. The backend waits for a short interval and checks the analysis status using the analysis ID.
7. If the analysis is completed, the result is fetched and parsed.
8. If the maximum polling limit is reached, the system returns a pending or delayed result.
9. The final verdict is displayed and its metadata is saved in file_scans.
10. Only a Malicious final verdict triggers an attempted malware email alert.

5.1.7 Scan History and Database Storage Algorithm

The history module presents the latest scan summaries across email, URL, and file modules. In the current prototype, the backend returns the most recent twenty system-wide scans rather than filtering by the logged-in user.

Algorithm Steps:

1. User performs an email, URL, or file scan.
2. The backend receives the result from the related detection module.
3. The related module inserts a record into email_scans, url_scans, or file_scans.
4. Each record stores user_email, module-specific summary and result fields, and scanned_at.
5. PDF report files and report paths are not stored in the database.
6. The history page requests the latest scan records.

7. The backend retrieves records from the database.
8. The frontend displays the scan history in tabular or card format.
9. The current history table does not provide record-detail or report-download actions.

5.1.8 Report Generation Algorithm

The report feature generates a PDF from the currently displayed result in the user browser using html2canvas and jsPDF. The generated file is downloaded locally and is not saved by the backend [30], [31].

Algorithm Steps:

1. User completes a scan.
2. User clicks the Download Report button.
3. The frontend collects the current scan result.
4. The captured result contains the scan type, verdict, confidence, indicators, and recommendations visible on screen.
5. The report is generated in PDF format.
6. The file is downloaded by the user.
7. No report record or report path is inserted into the database.

5.1.9 Admin Panel Workflow

The current admin panel is a read-only monitoring interface. It displays system statistics, registered users, recent activity, and the latest fifty scans; it does not edit users, settings, logs, or model files.

Algorithm Steps:

1. A user logs in with an email configured in the ADMIN_EMAILS environment variable.
2. The prototype backend compares the X-User-Email request header with that configured list.
3. After successful login, the admin dashboard is displayed.
4. The Overview tab shows users, scan counts, detected threats, and recent activity.
5. The Users tab shows registration dates and scan counts but provides no modification controls.
6. The Scans tab shows recent email, URL, and file scan summaries.
7. Statistics are calculated from current database records.
8. Signed tokens and server-enforced role records are future security work.

5.1.10 Email Notification Workflow

The notification module sends welcome messages after registration and attempts alerts for configured high-risk scan outcomes. Missing SMTP configuration, anonymous requests, or delivery failures do not interrupt the scan.

Algorithm Steps:

1. A scan is completed.
2. The system checks the final threat level.
3. Alerts are attempted for spam predictions above 0.70, malicious URLs above 0.60 confidence, and Malicious file verdicts with at least one detection.
4. The system prepares an email message containing scan type, threat verdict, confidence score, short explanation, and safety recommendation.
5. The email is sent to the registered user using SMTP or configured email service.
6. Delivery success or failure is written to application logs; there is no notification database table.
7. If email sending fails, the system records the error and continues without interrupting the scan result display.

5.1.11 Gmail/Chrome Extension Workflow

The Gmail/Chrome extension allows users to scan email content directly from Gmail. This makes the system more practical because users do not always need to copy and paste email text manually.

Algorithm Steps:

1. User opens Gmail in the Chrome browser.
2. User opens a suspicious email.
3. User clicks the CyberSentinel AI extension icon.
4. The content script reads the largest visible Gmail message-text block.
5. The extension submits up to 5,000 characters to the live /predict endpoint.
6. The backend runs phishing detection and embedded link analysis.
7. The backend returns the verdict and confidence score.
8. The extension popup displays the result to the user.
9. If the email is risky, the user receives a warning.
10. Extension requests do not currently include an authenticated user header and are stored as anonymous if database saving succeeds.

5.1.12 Live Deployment Workflow

The final version of CyberSentinel AI is deployed so that it can be accessed beyond the local development environment.

Algorithm Steps:

1. The Flask backend is deployed on Render.
2. Environment variables are configured securely on the hosting platform.
3. API keys are stored on the server side only.
4. The React frontend is deployed on Vercel.
5. Frontend API base URL is changed from localhost to the live backend URL.
6. CORS settings are updated to allow the deployed frontend domain.
7. PostgreSQL is configured for persistent deployed storage, while SQLite remains available locally.
8. All modules are tested from a different device and network.
9. The Gmail extension is connected to the live backend.

5.2 External APIs

CyberSentinel AI integrates VirusTotal for file reputation and live analysis, AbuseIPDB for resolved-IP reputation, and SMTP for welcome and high-risk alert email. API credentials remain in backend environment variables and are not embedded in frontend or extension code [14], [15].

Table 5. 1: Details of APIs Used in the Project

Name of API	Description of API	Purpose of Usage	Function/Class Name in Which It Is Used
VirusTotal API v3	VirusTotal is a threat intelligence platform that analyzes files and URLs using multiple antivirus engines and security vendors.	Used for malware file scanning, SHA-256 hash lookup, live upload of unknown files, and retrieval of file analysis results.	MalwareScanner, malware_service.py, file_scan_route
AbuseIPDB API	AbuseIPDB provides reputation information about IP addresses reported for malicious or abusive activity.	Used to check the reputation of IP addresses resolved from submitted URLs.	URLAnalyzer, url_service.py, abuseipdb_lookup
SMTP / Email Service	Configured SMTP service for welcome messages and selected high-risk alerts.	Used after signup and after configured phishing, URL, or malware outcomes.	notification_service.py, send_welcome_email(), send_threat_alert()
CyberSentinel Backend API	Internal REST API developed using Flask for communication between the frontend, extension, and backend modules.	Used by React frontend and Gmail extension to submit scans and receive results.	app.py, predict_routes.py, url_routes.py, malware_routes.py
Gmail / Chrome Extension API	Browser extension interface used to interact with Gmail page content and browser popup components.	Used to extract email text and links from Gmail and send them to the CyberSentinel live backend.	manifest.json, content.js, popup.js

These integrations complement the local phishing model and deterministic URL rules. External failures are handled with error, unavailable, pending, or fallback responses instead of being presented as successful threat-intelligence checks.

5.3 User Interface

The user interface of CyberSentinel AI is designed to be simple, clear, and understandable for non-technical users. Since the target users include students, freelancers, small business owners, and general internet users, the interface avoids unnecessary technical complexity and presents results in plain language.

The system uses a dashboard-based interface developed in React.js. Users can navigate to different modules from the sidebar or navigation menu. Each module follows a simple workflow: submit input, click scan, view result, and download report if required.

5.3.1 Login and Registration Interface

The login and registration interface supports account creation and prototype login state. The current frontend stores a dummy token, email address, and login flag in localStorage; production-grade signed sessions and role records are future work.

Main features include:

- User registration
- User login
- Configured admin-email check for the prototype admin view
- Client-side login-state handling
- Logout functionality

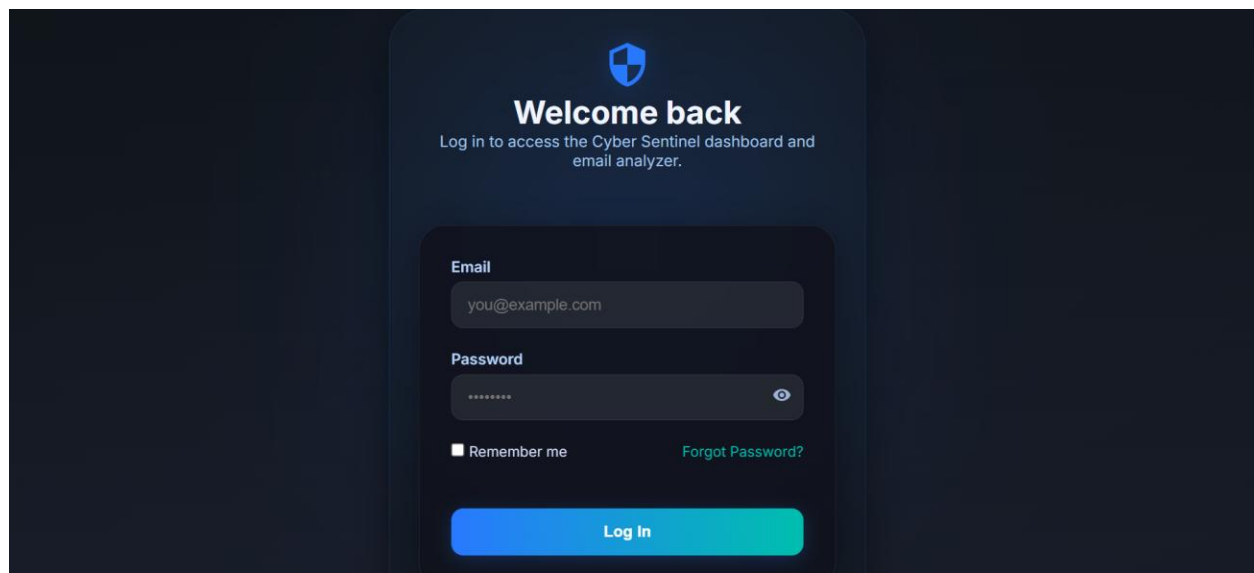


Figure 5. 1: Login and Registration Interface

5.3.2 User Dashboard

The dashboard links to the three scanning modules and displays database-backed email-scan and threat counts from the public /api/stats endpoint, refreshed every thirty seconds.

This section shows:

- Email phishing scan
- URL threat analysis
- Malware file scan
- Scan history
- Database-backed system statistics
- Access to scan-history and admin-monitoring views
- Logout

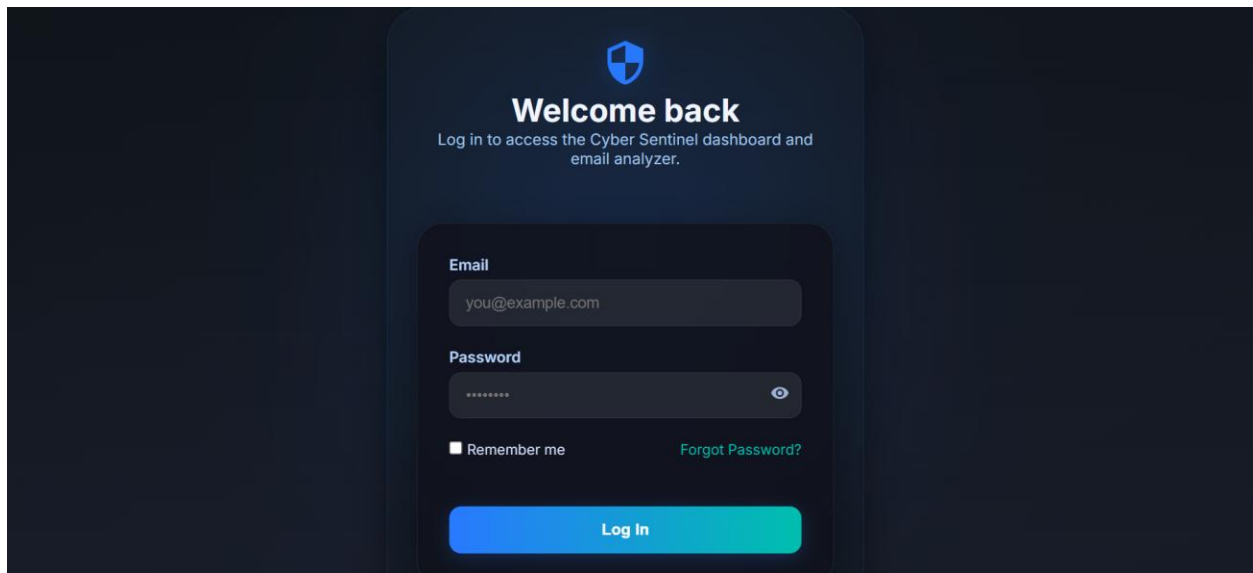


Figure 5. 2: User Dashboard

5.3.3 Email Phishing Scan Interface

The email scan page accepts pasted text, TXT/EML email files, and PDF/DOCX documents. The binary model result is displayed as Safe or Phishing with confidence, indicators, recommendations, and any visible embedded-link results.

Main features include:

- Threat verdict
- Confidence score
- Explanation
- Embedded link analysis
- TXT/EML upload and PDF/DOCX text extraction
- Recommended action

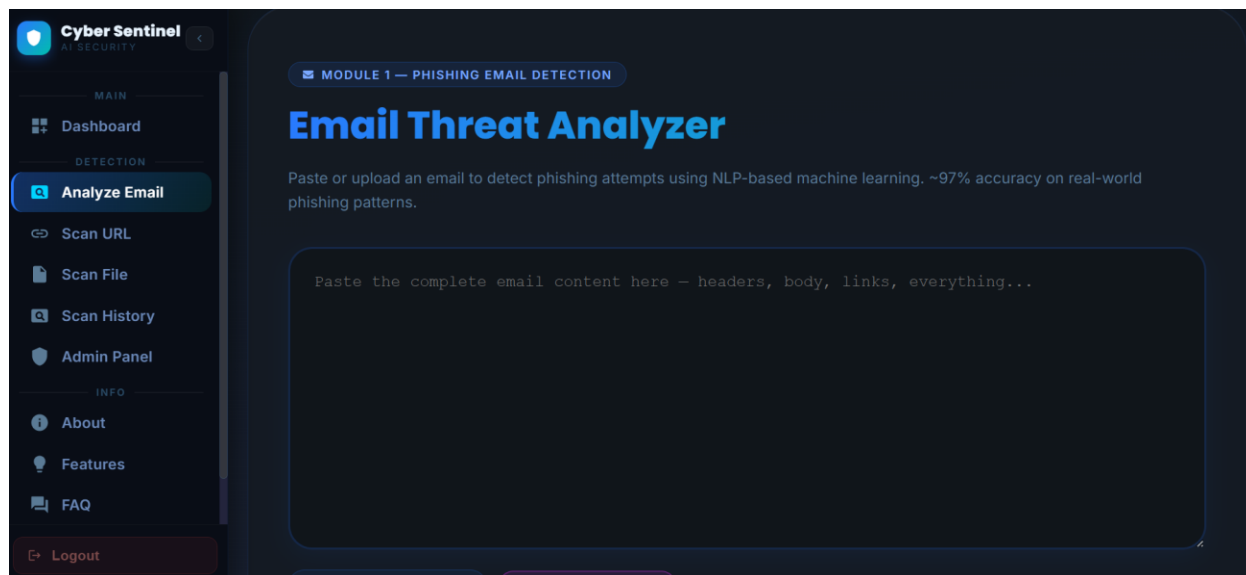


Figure 5. 3: Email Phishing Scan Interface

5.3.4 Email Result and Embedded URL Interface

The result section displays the Safe or Phishing verdict, confidence, threat indicators, safety recommendations, and separately analyzed visible HTTP/HTTPS URLs when any are found. Hidden HTML href mismatch detection is not implemented in the current version.

This section shows:

- Email verdict and confidence

- Visible extracted URL, when present
- Individual URL verdict and risk score
- Threat indicators and safety recommendations
- Scan-again and PDF-download actions

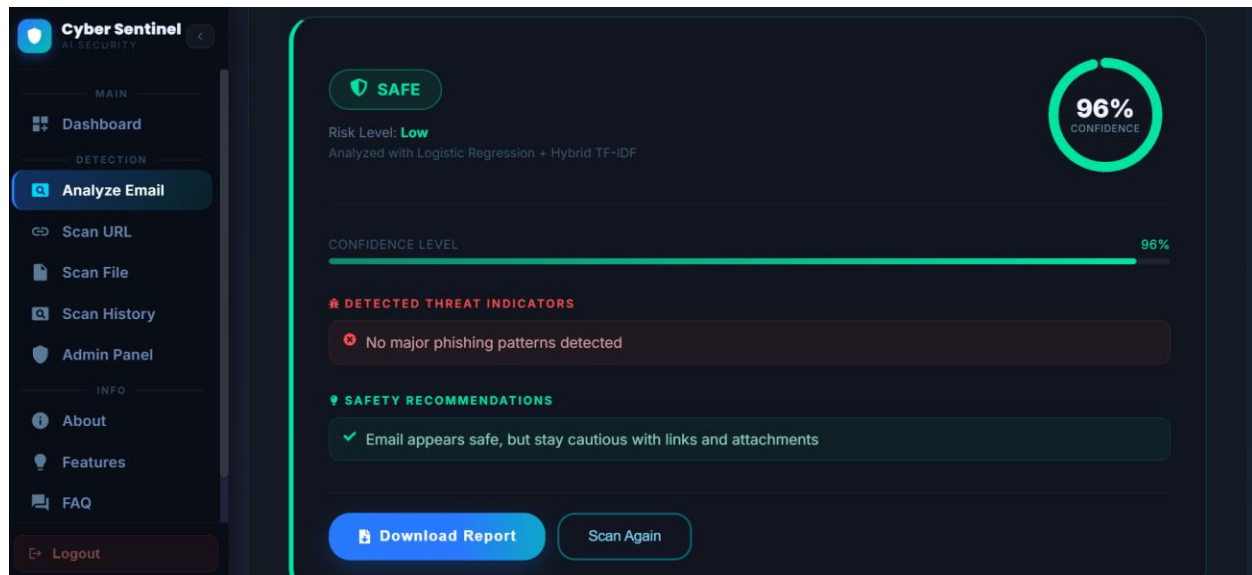


Figure 5. 4: Email Phishing Scan Result

5.3.5 URL Threat Analysis Interface

The URL scan page applies ten deterministic structural checks and, when available, AbuseIPDB reputation for the resolved IP address.

Main features include:

- URL verdict
- Risk score
- Suspicious features
- AbuseIPDB result
- AbuseIPDB availability and abuse score
- Final recommendation

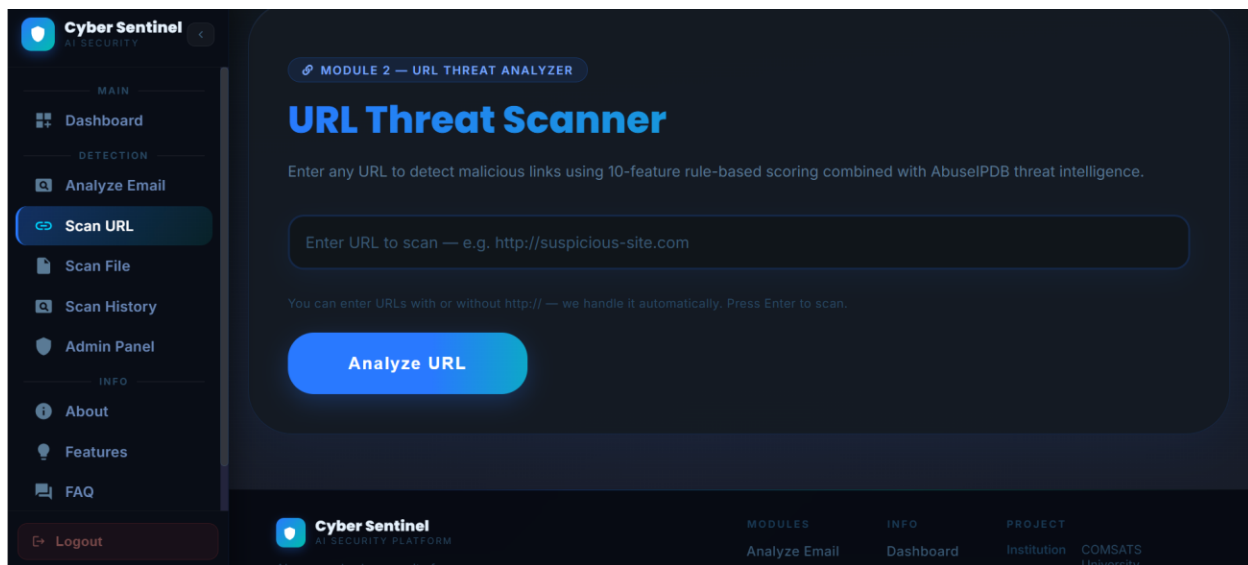


Figure 5. 5: URL Threat Analysis Interface

5.3.6 Malware File Scan Interface

The file scan page validates supported extensions and a 32 MB limit, creates a temporary backend file, and uses VirusTotal hash lookup or live upload. The temporary file is deleted, but unknown files may be transmitted to VirusTotal for analysis.

Main features include:

- File name
- File hash
- VirusTotal result
- Malicious engine count
- Suspicious engine count
- Final verdict
- Temporary-file deletion status and clear external-upload disclosure

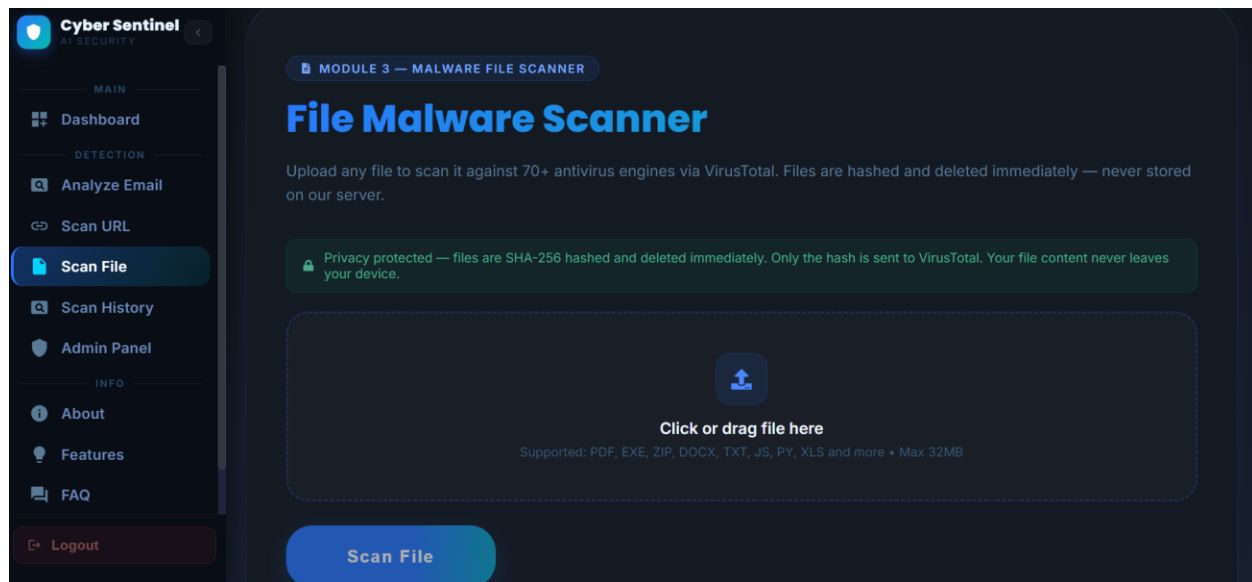


Figure 5. 6: Malware File Scan Interface

5.3.7 Scan History Interface

The history page displays the latest twenty scan summaries across all modules. The current backend response is system-wide and the table does not provide record-detail or report-download actions.

Main features include:

- Scan type
- Date and time
- Threat verdict
- Confidence score
- Module-specific confidence where available
- System-wide latest-record limitation

Type	Summary	Result	Confidence	Time
URL	https://mail.google.com/mail/u/1/#all	Safe	85%	7/6/2026, 3:21:54 PM
EMAIL	Dear JazzCash customer your account has been suspe...	Spam	98%	7/6/2026, 3:20:22 PM
EMAIL	[Word] phishing text.docx: Dear JazzCash customer ...	Spam	98%	7/6/2026, 3:15:37 PM
EMAIL	[Word] phishing text.docx: Dear JazzCash customer ...	Spam	98%	7/6/2026, 2:54:27 PM
EMAIL	Dear JazzCash customer your account has been suspe...	Spam	98%	7/6/2026, 2:49:04 PM
EMAIL	[Word] phishing text.docx: Dear JazzCash customer ...	Spam	98%	7/6/2026, 4:53:55 AM
EMAIL	[Word] arabic_assignment.docx: ال نوافل الخمار في ال...	Spam	89%	7/6/2026, 4:52:13 AM
EMAIL	[Word] CyberSentinel AI SRS.docx: COMSATS Universi...	Spam	97%	7/6/2026, 4:51:44 AM
EMAIL	hello my name is mohavia	Ham	97%	7/5/2026, 10:28:44 AM
FILE	arabic_assignment.docx	Clean	-	6/27/2026, 5:39:12 AM
FILE	arabic_assignment.docx	Pending	-	6/27/2026, 5:37:03 AM
FILE	arabic_assignment.docx	Unknown	-	6/27/2026, 5:36:03 AM

Figure 5. 7: Scan History Interface

5.3.8 Report Interface

Each result page can capture the displayed result and download it as a PDF through client-side libraries. Reports are not persisted by the backend.

Main features include:

- Project name
- Scan type
- Verdict
- Confidence score
- Threat details
- Safety tips
- Local browser download

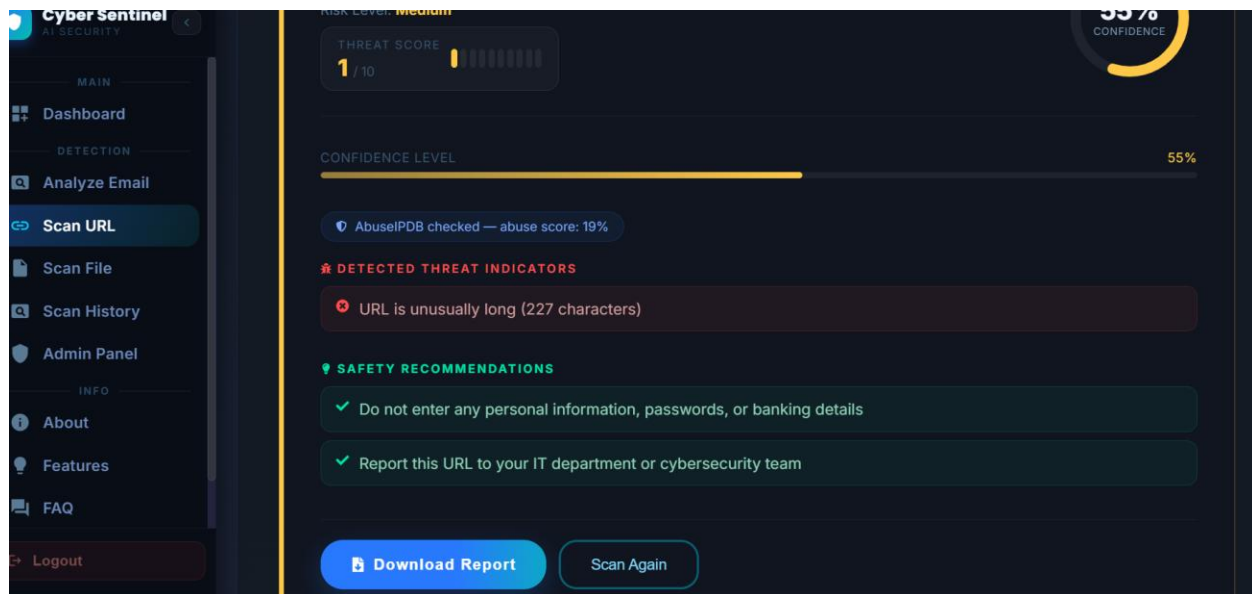


Figure 5. 8: PDF Report Generation Option

5.3.9 Admin Panel Interface

The admin panel provides read-only monitoring for configured administrator emails. It exposes Overview, Users, and Scans tabs backed by database queries.

Main features include:

- View total users
- View total scans
- View detected threats
- View registered users and per-module scan counts
- View recent scan summaries
- Refresh current database statistics
- No user-editing, log-management, settings, or model controls

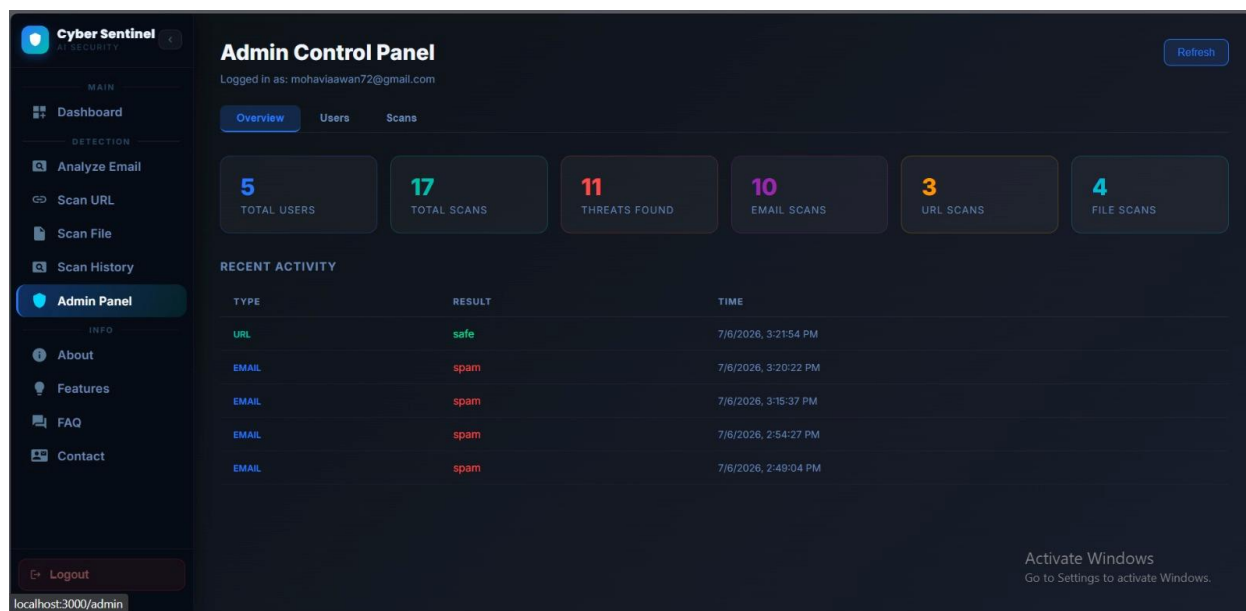


Figure 5. 9: Admin Panel Interface

5.3.10 Email Notification Output

Email output is generated for welcome messages and selected high-risk results when SMTP credentials are configured. The example shown is a high-confidence phishing alert.

This section shows:

- Scan type
- Threat level
- Confidence score
- Short explanation
- Recommended user action

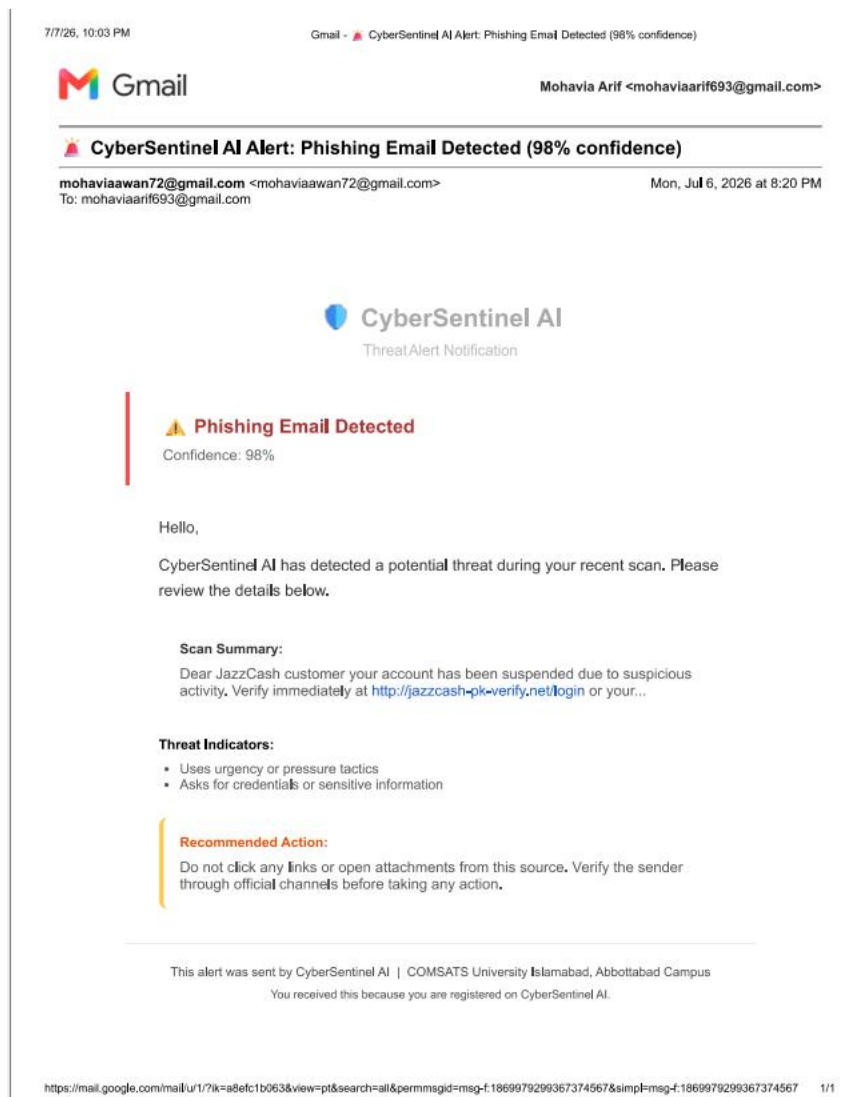


Figure 5. 10: Email Notification Output

5.3.11 Gmail/Chrome Extension Interface

The Gmail extension reads the visible text of an opened Gmail message and sends it to the live phishing endpoint. The popup displays a binary verdict, confidence, threat indicators, and any visible URLs returned by the backend.

Main features include:

- Scan current email button
- Result badge
- Confidence score
- Risk explanation

- Link warning

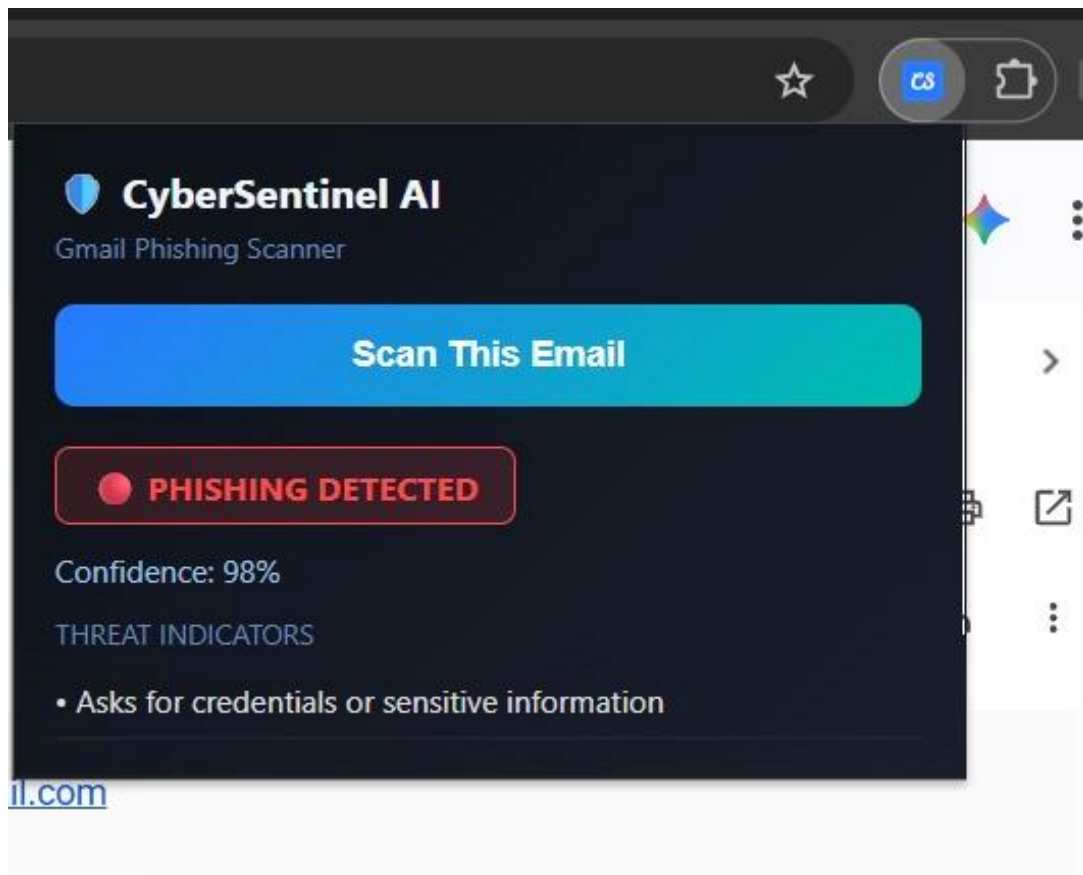


Figure 5. 11: Gmail Extension Interface

5.4 Summary

This chapter described the implemented ML email classifier, document-text extraction, visible embedded-link analysis, deterministic URL scoring, VirusTotal workflow, PostgreSQL/SQLite storage, client-side reports, prototype administration, notifications, Gmail extension, and deployed architecture without attributing unimplemented features to the system.

The next chapter discusses the testing and evaluation of the implemented system, including manual testing, unit testing, functional testing, integration testing, and automated testing.

6. Testing and Evaluation

This chapter presents the testing and evaluation process performed on CyberSentinel AI - Intelligent Cybersecurity Toolkit for Phishing, Malware, and Threat Detection. Testing was conducted to verify that all implemented modules work according to the defined requirements and that the system performs reliably in real usage conditions. The testing process included manual testing, system testing, unit testing, functional testing, integration testing, automated API testing, deployment testing, and extension testing.

The final version includes binary phishing detection, visible embedded-link scanning, TXT/EML/PDF/DOCX input, rule-based URL analysis, VirusTotal file lookup and live upload, latest scan history, client-side PDF reports, read-only admin monitoring, configured email notifications, a Gmail extension, and live deployment.

Testing was performed to ensure that the system correctly accepts user input, validates data, processes requests through the backend, performs AI/ML or API-based analysis, stores results in the database, generates reports, sends notifications, and displays results clearly to the user. The main goal of testing was to confirm that CyberSentinel AI is accurate, usable, secure, and reliable for non-technical users.

6.1 Manual Testing

Manual testing was performed by interacting with the system through the user interface and verifying whether each feature behaved as expected. The system was tested from the perspective of both normal users and administrators. Manual testing helped identify interface issues, incorrect messages, missing validations, and integration problems that may not be detected through automated tests.

The following modules were manually tested:

- User registration and login
- Email phishing scan
- Embedded link scanning
- Email and document file upload
- URL threat analysis
- Malware file scanning
- VirusTotal unknown file upload
- Scan history
- PDF report generation
- Email notifications

- Admin dashboard
- Gmail/Chrome extension
- Live deployment access

6.1.1 System Testing

System testing was performed after all modules were integrated into one complete platform. The purpose of system testing was to verify that CyberSentinel AI works as a complete cybersecurity toolkit rather than separate individual modules. The testing checked complete user journeys from login to scan submission, result display, report generation, scan history storage, and notification delivery.

Table 6. 1: System Testing

No.	Test Case / Test Script	Input / Attribute	Expected Result	Result
1	Open the deployed CyberSentinel AI web application	Live frontend URL	Homepage or login page loads successfully	Pass
2	Register a new user account	Email and password	User account is created successfully	Pass
3	Login with valid user credentials	Registered email and password	User dashboard is displayed	Pass
4	Login with invalid credentials	Wrong email or password	Error message is displayed	Pass
5	Submit phishing email text	JazzCash-style phishing message	System returns spam and the interface displays Phishing	Pass
6	Submit safe email text for scanning	Normal meeting email	System classifies email as safe	Pass
7	Submit email containing visible URL	Email with suspicious URL	Email result and embedded URL result are displayed	Pass
8	Upload a PDF or DOCX containing email text	Readable document	Text is extracted and classified	Pass

9	Submit suspicious URL	URL containing suspicious keywords and long path	URL is classified as suspicious or malicious	Pass
10	Submit safe URL	Trusted website URL	URL is classified as safe	Pass
11	Upload known malware test file/hash sample	Supported file type	VirusTotal result is displayed	Pass
12	Upload unknown clean Word file	New .docx file	File is uploaded to VirusTotal for live analysis and result is returned	Pass
13	Generate scan report	Completed scan result	PDF report is generated and downloaded	Pass
14	View scan history	Existing scan records	Latest system-wide scan summaries are displayed	Pass
15	Admin login	Admin credentials	Admin dashboard is displayed	Pass
16	View admin statistics	Admin dashboard	Total scans, users, and threats are displayed	Pass
17	Trigger configured risk notification	High-confidence phishing, malicious URL, or malicious file	SMTP alert is attempted without blocking the scan	Pass
18	Scan Gmail message using extension	Open Gmail email and click extension	Extension displays scan verdict	Pass
19	Test system from another device/network	Live deployed link	System opens and scans successfully	Pass
20	Logout from system	Logout button	User session ends successfully	Pass

System testing confirmed that the complete platform works according to the expected workflow and that all major modules are connected properly.

6.1.2 Unit Testing

Unit testing was performed on individual modules to verify that each component works correctly before integration. Each unit was tested separately using controlled input and

expected output. Unit testing helped ensure that errors inside one module did not affect the rest of the system.

Unit Testing 1: Phishing Email Detection

Testing Objective: To ensure that the phishing detection model correctly classifies email text.

Table 6. 2: Phishing Email Detection Unit Test Case

No.	Test Case / Test Script	Attribute and Value	Expected Result	Result
1	Test phishing email classification	Your JazzCash account is suspended. Verify now.	Email classified as spam / displayed as Phishing	Pass
2	Test safe email classification	Project meeting is scheduled for Monday.	Email classified as safe	Pass
3	Test empty email input	Empty text field	Error message displayed	Pass
4	Test long email input	Large email text	System processes or rejects safely according to limit	Pass
5	Test obfuscated phishing words	verif1cation required for acc0unt	Suspicious pattern detected	Pass

Unit Testing 2: Embedded Link Scanning

Testing Objective: To ensure that visible URLs inside email text are extracted and analyzed separately.

Table 6. 3: Embedded Link Scanning Unit Test Case

No.	Test Case / Test Script	Attribute and Value	Expected Result	Result
1	Extract one URL from email	Email contains one suspicious link	URL is extracted successfully	Pass
2	Extract multiple URLs	Email contains several URLs	Up to five unique URLs are analyzed	Pass
3	Remove duplicate URLs	Same URL repeated twice	Duplicate URL is ignored	Pass
4	Scan extracted suspicious URL	Extracted phishing-style URL	URL verdict shown separately	Pass
5	Submit email without URLs	Email contains no URL	System displays email result without embedded link section	Pass

Unit Testing 3: Email and Document File Upload

Testing Objective: To verify TXT/EML decoding, PDF/DOCX text extraction, and unsupported or unreadable-file handling.

Table 6. 4: Email and Document File Upload Unit Test Case

No.	Test Case / Test Script	Attribute and Value	Expected Result	Result
1	Upload TXT email file	Readable UTF-8 text	Content decoded and classified	Pass
2	Upload EML file	Readable email source	File treated as decoded text and classified	Pass
3	Upload text-based PDF	PDF containing email text	Page text extracted and classified	Pass
4	Upload DOCX document	DOCX containing email text	Paragraph text extracted and classified	Pass
5	Upload unreadable or unsupported file	Image-only/empty/unsupported input	Clear validation or extraction error	Pass

Unit Testing 4: URL Threat Analyzer

Testing Objective: To ensure that URL analysis correctly checks structural features and API reputation.

Table 6. 5: URL Analyzer Unit Test Case

No.	Test Case / Test Script	Attribute and Value	Expected Result	Result
1	Test safe HTTPS URL	https://www.comsats.edu.pk	URL classified as safe	Pass
2	Test HTTP URL	http://example.com/login	HTTP risk feature detected	Pass
3	Test IP-based URL	http://192.168.1.1/login	IP-domain risk detected	Pass
4	Test suspicious keyword URL	URL containing verify-account-login	Suspicious keyword risk detected	Pass

5	Test shortened URL	Shortener-based URL	URL shortener warning displayed	Pass
6	Test AbuseIPDB lookup	URL domain resolved to IP	IP reputation checked successfully	Pass

Unit Testing 5: Malware File Scanner

Testing Objective: To ensure that file validation, hash generation, VirusTotal lookup, and live upload work correctly.

Table 6. 6: Malware File Scanner Unit Test Case

No.	Test Case / Test Script	Attribute and Value	Expected Result	Result
1	Upload supported file type	.pdf, .docx, .txt, or .zip	File accepted for scanning	Pass
2	Upload unsupported file type	Unsupported extension	Error message displayed	Pass
3	Upload large file	File exceeding size limit	File rejected safely	Pass
4	Generate SHA-256 hash	Valid uploaded file	Correct hash generated	Pass
5	VirusTotal hash lookup	Known file hash	VirusTotal report returned	Pass
6	Unknown file live upload	New file not found by hash lookup	File uploaded for live analysis	Pass
7	Poll VirusTotal analysis	Analysis ID returned	Final analysis result retrieved	Pass
8	Delete temporary file	File after scan	Temporary file removed	Pass

Unit Testing 6: Authentication, Database, and Scan History

Testing Objective: To verify account lookup, prototype login responses, database inserts, and latest-history retrieval.

Table 6. 7: Authentication and Database Unit Test Case

No.	Test Case / Test Script	Attribute and Value	Expected Result	Result
1	Register new user	Valid email and password	User record stored	Pass
2	Prevent duplicate registration	Existing email	Duplicate account error displayed	Pass
3	Login valid user	Correct credentials	Success response and prototype token returned	Pass
4	Login invalid user	Wrong credentials	Login rejected	Pass

5	Store email scan result	Completed email scan	Record saved in database	Pass
6	Store URL scan result	Completed URL scan	Record saved in database	Pass
7	Store file scan result	Completed file scan	Record saved in database	Pass
8	Retrieve scan history	History request	Latest system-wide records returned	Pass

Unit Testing 7: Admin Panel and Email Notifications

Testing Objective: To verify read-only admin monitoring and configured notification behavior.

Table 6. 8: Admin Panel and Email Notification Unit Test Case

No.	Test Case / Test Script	Attribute and Value	Expected Result	Result
1	Login as admin	Admin credentials	Admin dashboard opens	Pass
2	View user list	Admin panel	Registered users displayed	Pass
3	View recent scans	Admin Scans tab	Latest scan summaries displayed	Pass
4	View threat statistics	Admin dashboard	Scan and threat statistics shown	Pass
5	Trigger threat notification	Configured high-risk result	Email is attempted when SMTP is configured	Pass
6	Handle email failure	Invalid SMTP setting or unreachable service	Error logged without crashing system	Pass

6.1.3 Functional Testing

Functional testing was performed to verify that each functional requirement defined in Chapter 3 is implemented correctly. The purpose of functional testing is to ensure that every module performs its required operation from the user perspective.

Table 6. 9: Functional Testing

No.	Functional Requirement	Test Case / Test Script	Expected Result	Result
1	User registration	User creates new account	Account is created successfully	Pass

2	User login	User enters valid credentials	Dashboard opens	Pass
3	Email scan	User submits email text	Email verdict is displayed	Pass
4	Phishing classification	Submit phishing-style email	System identifies risk	Pass
5	Embedded link scanning	Submit email containing URL	Extracted link is scanned separately	Pass
6	Email/document file upload	Upload TXT, EML, PDF, or DOCX	Readable text is classified	Pass
7	URL analysis	User submits suspicious URL	URL risk result displayed	Pass
8	AbuseIPDB integration	Submit URL with resolvable domain	IP reputation is checked	Pass
9	File scanning	User uploads file	Malware scan result displayed	Pass
10	VirusTotal live upload	Upload unknown file	File is uploaded and analyzed	Pass
11	Result visualization	Complete any scan	Green/yellow/red result indicator shown	Pass
12	Scan history	Open history page	Latest system-wide scans are displayed	Pass
13	PDF report	User downloads report	Report downloaded successfully	Pass
14	Admin monitoring	Configured admin opens panel	Statistics, users, and scans load	Pass
15	Email notification	Configured high-risk result	Alert is attempted without blocking scan	Pass
16	Gmail extension	Scan opened Gmail email	Extension shows verdict	Pass
17	Logout	User logs out	Session is cleared	Pass

Functional testing confirmed that all major user-facing features of CyberSentinel AI are working as expected.

6.1.4 Integration Testing

Integration testing was performed to verify communication between different modules. Since CyberSentinel AI includes frontend, backend, machine learning model, database, external APIs, email service, and browser extension, integration testing was important to ensure that all components work together correctly.

Table 6. 10: Integration Testing

No.	Integrated Modules	Test Case / Test Script	Expected Result	Result
1	React frontend + Flask backend	Submit email scan request	Backend receives request and returns result	Pass
2	Backend + phishing ML model	Send preprocessed email text to model	Model returns prediction and confidence	Pass
3	Email scan + URL analyzer	Submit email containing URL	Email and embedded link results are returned together	Pass
4	Document extractor + phishing model	Upload PDF or DOCX	Readable text is extracted and classified	Pass
5	URL analyzer + AbuseIPDB	Submit URL for scanning	IP reputation result included	Pass
6	File scanner + VirusTotal	Upload known file	VirusTotal report returned	Pass
7	File scanner + VirusTotal live upload	Upload unknown file	Live analysis result returned after polling	Pass
8	Backend + database	Complete any scan	Result stored in database	Pass
9	Frontend + scan history API	Open history page	Latest records across modules are displayed	Pass
10	Client report generator + scan result	Download PDF	Current result is captured and downloaded locally	Pass
11	Scan module + email service	Suspicious result generated	Notification email sent	Pass
12	Admin panel + database	Open admin tabs	Statistics, users, and scans load	Pass
13	Gmail extension + live backend	Scan visible Gmail text	Binary prediction returned to popup	Pass
14	Live frontend + live backend	Access from different network	Complete scan works successfully	Pass

Integration testing confirmed that the system modules communicate correctly and that the final deployed system works as a complete platform.

6.2 Automated Testing

Python integration scripts and Postman requests were used to exercise core backend endpoints, validation, and response structures. These scripts are useful regression checks but are not a comprehensive pytest or unittest suite.

Additional verification used browser developer tools, database inspection, Chrome extension developer mode, and deployment logs. The React repository still contains the default Create React App test and does not provide substantive automated frontend coverage.

6.2.1 Tools Used

Table 6. 11: Tools Used for Automated Testing

Tool Name	Tool Description	Applied On	Results
Postman	API testing tool used to send HTTP requests and inspect responses.	Login API, email scan API, URL scan API, file scan API, scan history API	APIs returned expected JSON responses
Python Test Scripts	Custom scripts used to test backend endpoints and prediction responses.	Phishing prediction, URL analyzer, malware scanner	Core backend functions tested successfully
Python integration scripts	Custom request scripts for endpoint and validation checks.	Core backend endpoints	Useful regression checks; not a formal pytest suite
Browser Developer Tools	Used to inspect frontend requests, console errors, and network responses.	React frontend and API integration	Frontend-backend communication verified
Database inspection	SQLite Browser locally and deployment database queries.	users and three scan tables	Records stored and retrieved
Chrome Extension Developer Mode	Used to load and test the Gmail/Chrome extension.	manifest.json, content script, popup interface	Extension installed and scanned Gmail content
Render logs	Deployment logs used to inspect backend behavior.	Live Flask backend	Backend reachable and errors inspectable
Vercel logs	Frontend deployment logs and network inspection.	Live React frontend	Frontend connected to backend
Email Test Inbox	Used to verify delivery of system notifications.	Email notification module	Notification emails received successfully

6.2.2 Automated API Test Results

Table 6. 12: Automated API Test Results

No.	API / Module Tested	Test Input	Expected Response	Result
1	Login API	Valid email and password	Success response with session/token	Pass
2	Login API	Invalid password	Error response	Pass

3	Email Scan API	Phishing email text	Prediction, confidence, explanation	Pass
4	Email Scan API	Empty text	Validation error	Pass
5	Embedded Link API Logic	Email containing URL	Embedded link result list	Pass
6	Email File Upload API	Valid TXT/EML file	Decoded text and prediction returned	Pass
7	URL Scan API	Suspicious URL	URL verdict and risk score	Pass
8	URL Scan API	Invalid URL	Validation error	Pass
9	File Scan API	Valid uploaded file	File scan result	Pass
10	File Scan API	Unsupported file	File type error	Pass
11	VirusTotal Upload Logic	Unknown file	Live analysis request created	Pass
12	Scan History API	GET request	Latest system-wide scan records returned	Pass
13	Admin API	Admin request	System statistics returned	Pass
14	Notification Service	Risky scan result	Email sent or logged	Pass
15	Extension Backend Request	Gmail email content	Prediction returned to extension	Pass

6.2.3 Model and Performance Evaluation

The saved binary Logistic Regression model was evaluated by recreating the stratified 80/20 split of phishing_clean_balanced.csv with random_state=42. On 1,333 held-out records it achieved 98.05% accuracy. Spam precision was 99.02%, recall 94.82%, and F1-score 96.88%; the confusion matrix contained 904 true ham, 4 ham classified as spam, 403 true spam, and 22 spam classified as ham. API response times remain dependent on network and external-service availability.

Table 6. 13: Model and Performance Results

No.	Test Area	Expected Performance	Observed Result	Result
1	Email phishing prediction	Result within a few seconds	Result returned quickly	Pass
2	Embedded link extraction	Links extracted immediately	Links extracted successfully	Pass
3	URL analysis without API delay	Result within a few seconds	Result returned successfully	Pass
4	AbuseIPDB lookup	Result depends on API response	Result returned or graceful fallback applied	Pass
5	File hash generation	Hash generated quickly for allowed file size	Hash generated successfully	Pass
6	VirusTotal known file lookup	Result depends on API response	Result returned successfully	Pass

7	VirusTotal unknown file upload	Longer due to upload and polling	Result returned after polling	Pass
8	Scan history retrieval	Records load within acceptable time	History displayed successfully	Pass
9	Gmail extension scan	Result displayed in extension popup	Result displayed successfully	Pass
10	Final phishing model evaluation	Held-out 20% split	98.05% accuracy; spam F1 96.88%	Pass

6.2.4 Security Testing

Security testing covered sanitization, validation, rate limiting, payload controls, environment-based API secrets, CORS, and temporary-file deletion. Authentication and administrator authorization remain prototype controls and are identified as partial rather than production-grade protections [17], [18].

Table 6. 14: Security Test Results

No.	Security Feature	Test Case / Test Script	Expected Result	Result
1	Input sanitization	Submit script tags in email input	Script removed or neutralized	Pass
2	Empty input validation	Submit blank scan request	Request rejected with error	Pass
3	File size limit	Upload file above allowed size	File rejected	Pass
4	File type validation	Upload unsupported file type	File rejected	Pass
5	API key protection	Inspect frontend code	API keys not visible in frontend	Pass
6	CORS configuration	Request from unauthorized origin	Request blocked or restricted	Pass
7	Password storage	Inspect stored value	Plain password is not stored	Partial - SHA-256 is unsalted
8	Prototype admin access	Normal and configured-admin headers	Configured list is checked	Partial - header is client supplied
9	Temporary file deletion	Scan uploaded file	Temporary file removed after processing	Pass
10	Rate limiting	Send repeated scan requests	Excessive requests limited	Pass

6.3 Summary

Testing supports the implemented scanning, storage, reporting, notification, extension, and deployment workflows. False hidden-link, PhishTank, user-management, audit-log, and report-storage claims have been removed. Prototype authentication, global history visibility, and limited automated test coverage remain documented limitations.

The final system is suitable for an academic demonstration of a hybrid cybersecurity toolkit. Production use would require stronger identity controls, per-user data isolation, formal automated tests, operational monitoring, and additional privacy review.

7. Conclusion and Future Work

This chapter concludes the development and implementation of CyberSentinel AI - Intelligent Cybersecurity Toolkit for Phishing, Malware, and Threat Detection. It summarizes the achievements of the project, highlights the effectiveness of the implemented modules, and discusses possible future enhancements that can further improve the system. The project successfully demonstrates how Artificial Intelligence, Machine Learning, Natural Language Processing, external threat intelligence APIs, secure web technologies, and user-centered design can be combined to build a practical cybersecurity solution for non-technical users.

CyberSentinel AI provides one deployed interface for phishing-email classification, visible URL inspection, and VirusTotal-assisted file analysis. Supporting features include document-text extraction, latest scan history, browser-generated reports, read-only admin monitoring, configured notifications, and a Gmail extension.

7.1 Conclusion

CyberSentinel AI achieves its main objective as a unified and understandable cybersecurity prototype. It combines different techniques according to the threat type instead of inaccurately describing every module as machine learning.

The phishing module uses hybrid word and character TF-IDF with binary Logistic Regression. The model labels are ham and spam and are displayed as Safe and Phishing. On the recreated held-out split of the final balanced dataset, the saved model achieved 98.05% accuracy, with 99.02% spam precision and 94.82% spam recall.

The URL module analyzes length, keywords, IP-based hosts, scheme, shorteners, top-level domains, subdomains, path length, and related structural indicators. AbuseIPDB adds optional resolved-IP reputation; PhishTank is not integrated in the final system.

The malware file scanning module provides file-based threat detection using SHA-256 hashing and VirusTotal API integration. In the final implementation, the system was improved beyond simple hash lookup by adding live upload and polling for unknown files. This makes the scanner more practical because a new file that is not already present in VirusTotal can still be submitted for live analysis.

The email workflow extracts and analyzes visible HTTP/HTTPS URLs. TXT and EML files are treated as decoded text, while PDF and DOCX files use text extraction. Structured MIME parsing, HTML anchor extraction, and visible-text versus href mismatch detection remain future work.

The final version also includes prototype authentication, PostgreSQL-backed scan summaries, browser-generated reports, read-only admin monitoring, application logging,

configured email alerts, a Gmail extension, and Render/Vercel deployment. Reports and application logs are not stored as database entities.

Manual and script-based verification covered the implemented workflows. The current limitations include prototype token handling, header-based admin checks, global rather than per-user history, and limited formal automated test coverage.

Overall, CyberSentinel AI provides an accessible and practical cybersecurity solution for students, freelancers, small businesses, and general internet users. It simplifies threat detection by presenting results through clear verdicts, confidence scores, color-coded indicators, reports, and recommendations. The project demonstrates the successful application of AI, ML, NLP, web development, database systems, cybersecurity principles, and software engineering practices in solving a real-world problem.

7.2 Future Work

The deployed prototype is functional, but the following improvements are required for stronger accuracy, privacy, authorization, scalability, and production readiness.

7.2.1 Larger Pakistani Cyber Threat Dataset

In future, the phishing dataset can be expanded with more real-world Pakistani phishing emails, scam messages, fake banking alerts, mobile wallet fraud examples, and malicious URLs. A larger local dataset would improve detection accuracy against threats targeting Pakistani users. The dataset can also include Urdu and Roman Urdu phishing messages to make the system more effective for local communication patterns.

7.2.2 Deep Learning-Based Phishing Detection

The current phishing detection model uses hybrid TF-IDF and Logistic Regression because it is fast, interpretable, and suitable for real-time classification. In future, advanced deep learning models such as BERT, DistilBERT, LSTM, or transformer-based classifiers can be explored. These models may improve contextual understanding, especially for complex phishing emails that use indirect language or advanced social engineering.

7.2.3 Full Browser-Wide Extension

The Gmail extension can be expanded into a browser-wide security assistant. Future email work should also add standards-based MIME parsing, HTML anchor extraction, and visible-text versus destination-domain mismatch detection.

7.2.4 Mobile Application

A mobile application can be developed for Android and iOS users. Since many Pakistani users access digital banking, emails, and social media through mobile phones, a mobile version would make the system more accessible. The mobile app can include email scanning, URL scanning, file scanning, notifications, and report viewing.

7.2.5 Behavioral Malware Analysis

The current malware scanner uses VirusTotal and file analysis results. In future, behavioral malware analysis can be added using sandbox execution. This would allow the system to observe how a suspicious file behaves when executed in a controlled environment. Behavioral analysis can detect threats that may not be identified by static scanning alone.

7.2.6 Enterprise Dashboard and SIEM Integration

For organizational use, CyberSentinel AI can be extended with enterprise-level dashboards and integration with Security Information and Event Management systems. This would allow companies, universities, and small businesses to monitor threats, export logs, generate analytics, and integrate CyberSentinel AI with existing cybersecurity workflows.

7.2.7 Multi-Language Support

Future versions can support multiple languages, especially Urdu and Roman Urdu. This is important because many phishing messages in Pakistan use mixed English, Urdu, and Roman Urdu. Multi-language support would improve detection for local users and make the system more inclusive.

7.2.8 Two-Factor Authentication and Advanced Security

Authentication should replace the dummy client token and unsalted SHA-256 password hashing with signed server sessions or JWTs and a slow salted password hash such as Argon2 or bcrypt. Administrator roles must be verified server-side, and scan-history queries should be restricted to the authenticated user. Two-factor authentication, session expiry, and stronger password policies can then be added.

7.2.9 Continuous Model Retraining

A feedback-based retraining pipeline can be added in future. If users or admins mark a scan result as correct or incorrect, the feedback can be stored and later used to improve the model. This would allow the system to adapt to new phishing patterns over time.

7.2.10 Cloud Scalability and Load Balancing

The deployed system can be improved through cloud scaling, load balancing, containerization, and monitoring tools. These improvements would help CyberSentinel AI handle more users and maintain stable performance under higher traffic.

7.3 Summary

CyberSentinel AI has been developed as a deployed AI-assisted prototype with binary phishing detection, visible embedded-link analysis, rule-based URL scoring, VirusTotal-assisted file analysis, scan summaries, browser reports, read-only admin monitoring, notifications, and a Gmail extension. Future work includes structured EML analysis,

stronger authentication and privacy, formal automated testing, larger multilingual datasets, mobile access, behavioral malware analysis, and scalable monitoring.

8. References

The development of CyberSentinel AI - Intelligent Cybersecurity Toolkit for Phishing, Malware, and Threat Detection was supported by standard software engineering concepts, machine learning techniques, cybersecurity practices, official documentation, and external API references. The following sources were consulted during the design, development, implementation, testing, and documentation of the system.

1. R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, McGraw-Hill Education.
2. I. Sommerville, Software Engineering, Pearson Education.
3. C. D. Manning, P. Raghavan, and H. Schütze, Introduction to Information Retrieval, Cambridge University Press.
4. S. Bird, E. Klein, and E. Loper, Natural Language Processing with Python, O'Reilly Media.
5. T. M. Mitchell, Machine Learning, McGraw-Hill Education.
6. C. M. Bishop, Pattern Recognition and Machine Learning, Springer.
7. D. Jurafsky and J. H. Martin, Speech and Language Processing, draft edition.
8. F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.
9. Scikit-learn Developers, "Scikit-learn Documentation," <https://scikit-learn.org/>, Last accessed: July 2026.
10. Python Software Foundation, "Python Documentation," <https://docs.python.org/>, Last accessed: July 2026.
11. Pallets Projects, "Flask Documentation," <https://flask.palletsprojects.com/>, Last accessed: July 2026.
12. React Developers, "React Documentation," <https://react.dev/>, Last accessed: July 2026.
13. SQLite, "SQLite Documentation," <https://www.sqlite.org/docs.html>, Last accessed: July 2026.
14. VirusTotal, "VirusTotal API v3 Documentation," <https://developers.virustotal.com/>, Last accessed: July 2026.
15. AbuseIPDB, "AbuseIPDB API Documentation," <https://www.abuseipdb.com/api.html>, Last accessed: July 2026.
16. PhishTank, "PhishTank Developer Information," https://phishtank.org/developer_info.php, Last accessed: July 2026.

17. OWASP Foundation, “OWASP Web Security Testing Guide,” <https://owasp.org/www-project-web-security-testing-guide/>, Last accessed: July 2026.
18. OWASP Foundation, “OWASP Top Ten Web Application Security Risks,” <https://owasp.org/www-project-top-ten/>, Last accessed: July 2026.
19. NIST, “Cybersecurity Framework,” National Institute of Standards and Technology, <https://www.nist.gov/cyberframework>, Last accessed: July 2026.
20. IEEE, ISO/IEC/IEEE 29148: Systems and Software Engineering - Life Cycle Processes - Requirements Engineering, IEEE Standards Association.
21. Object Management Group, “Unified Modeling Language Specification,” <https://www.uml.org/>, Last accessed: July 2026.
22. GitHub Docs, “GitHub Documentation,” <https://docs.github.com/>, Last accessed: July 2026.
23. Postman, “Postman API Platform Documentation,” <https://learning.postman.com/docs/>, Last accessed: July 2026.
24. Google Chrome Developers, “Chrome Extensions Documentation,” <https://developer.chrome.com/docs/extensions/>, Last accessed: July 2026.
25. Mozilla Developer Network, “Web APIs and JavaScript Documentation,” <https://developer.mozilla.org/>, Last accessed: July 2026.
26. Flask-Limiter Documentation, “Rate Limiting for Flask Applications,” <https://flask-limiter.readthedocs.io/>, Last accessed: July 2026.
27. Python-dotenv Documentation, “Python-dotenv,” <https://pypi.org/project/python-dotenv/>, Last accessed: July 2026.
28. joblib Developers, “Joblib Documentation,” <https://joblib.readthedocs.io/>, Last accessed: July 2026.
29. SciPy Developers, “SciPy Documentation,” <https://docs.scipy.org/doc/scipy/>, Last accessed: July 2026.
30. jsPDF Developers, “jsPDF Documentation,” <https://github.com/parallax/jsPDF>, Last accessed: July 2026.
31. html2canvas Developers, “html2canvas Documentation,” <https://html2canvas.hertzen.com/>, Last accessed: July 2026.
32. Google, “Gmail Help and Gmail Security Information,” <https://support.google.com/mail/>, Last accessed: July 2026.
33. Render, “Render Documentation,” <https://render.com/docs>, Last accessed: July 2026.
34. Vercel, “Vercel Documentation,” <https://vercel.com/docs>, Last accessed: July 2026.

35. Netlify, “Netlify Documentation,” <https://docs.netlify.com/>, Last accessed: July 2026.
36. Kaggle, “Phishing Email Dataset Resources,” <https://www.kaggle.com/>, Last accessed: July 2026.
37. Cisco Talos Intelligence Group, “Talos Intelligence,” <https://talosintelligence.com/>, Last accessed: July 2026.
38. Kaspersky, “Threat Intelligence Portal,” <https://opentip.kaspersky.com/>, Last accessed: July 2026.
39. Google, “Google Safe Browsing,” <https://safebrowsing.google.com/>, Last accessed: July 2026.
40. Microsoft, “Microsoft Security Documentation,” <https://learn.microsoft.com/en-us/security/>, Last accessed: July 2026.